

Roni Purolainen

MONITEHTÄVÄOPPIMINEN

Apple Inc. pörssidataan sovellettuna

Informaatioteknologian ja viestinnän tiedekunta
Pro gradu -tutkielma
Heinäkuu 2025

Tiivistelmä

Roni Purolainen:
Pro gradu -tutkielma
Tampereen yliopisto
Statistical data-analytics
Heinäkuu 2025

Tämän työn tarkoituksena on tutkia monitehtäväoppimisen soveltamista pörssiaineistoon. Työssä vertaillaan viittä eri mallia, joista kolme on yhden tehtävän malleja ja kaksi monitehtävämalleja. Yhden tehtävän malleja ovat Tukivektoregressio (SVR), Pitkäkestoinen-lyhytkestomuisti-rakenne (LSTM) ja Transformer. Monitehtävämalleina on versiot LSTM ja Transformer malleista. Tukivektoregressiota käytetään tutkimuksessa vertailukohtana muille malleille. Kaikki mallit pyrkivät mallintamaan Apple osakkeen hinnan logaritmista palautusarvoa. Monitehtävämallit eroavat yhden tehtävien malleista siten, että niissä on yksi ylimääräinen tehtävä ja kaksi täysin kytettyä kerrosta lisää yhden tehtävän malliin verrattuna. Monitehtävämalleissa toinen tehtävä on Keskiarvoinen todellinen vaihtelu (ATR).

Mallien tuloksista huomataan, että yhden tehtävän mallit eivät pärjänneet vertailukohta mallia paremmin, vaan yli sovittivat helposti ja antoivat epäluotettavia tuloksia. Toisin kuin yhden tehtävän mallit, monitehtävämallit onnistuivat paremmin ja niiden virhearvot putosivat huomattavasti SVR mallin virheiden alle. Lisäksi monitehtävämallien koulutus oli tasaisempaa, eivätkä mallit ylisovittaneet koulutuksen aikana. Yhden tehtävän malleissa virhe alkoi koulutuksen aikana kasvaa, kun taas monitehtävämalleilla virhe jatkoi kulkua alaspäin. Parhaan tuloksen antoi monitehtävä Transformer.

Malleille suoritettiin myös niin sanottu aikainen pysäytys, jossa koulutus lopetetaan virheen kasvaessa. Koulutusdatan viimeiset 1000 tapahtumaa on otettu niin sanottuun validaatio dataan, jolla virheen suuruutta arvioidaan. Aikaisen pysäytyksen mallit suoriutuivat kuitenkin kaikki heikommin kuin alkuperäiset mallit. Tämä viittaa siihen, että viimeiset 1000 tapahtumaa sisältävät mallille tärkeitä informaatiota. Myös tässä tapauksessa monitehtävämallit pärjäsivät yhden tehtävän malleja paremmin ja Transformer antoi parhaat tulokset.

Avainsanoja: monitehtäväoppiminen, neuroverkko, tukivektoregressio, pitkäkestoinen-lyhytkestomuisti-rakenne

Tämän julkaisun alkuperäisyys on tarkastettu Turnitin Originality Check -ohjelmalla.

Sisällys

1	Johdanto	4
2	Kirjallisuuskatsaus	6
3	Työssä käytetyt menetelmät	7
3.1	Koulutus ja testaus	7
3.2	Tukivektorikone (SVM)	8
3.3	Tukivektoriregressio (SVR)	8
3.4	Keinotekoiset neuroverkot (ANN)	9
3.4.1	Yhden neuronin perseptron	10
3.4.2	Moni kerroksinen perseptroni	10
3.4.3	Vastavirta-algoritmi	11
3.5	Takaisinkytkety neuroverkko (RNN)	13
3.6	Pitkäkestoinen-lyhytkestomuisti-rakenne (LSTM)	14
3.7	Transformer	15
3.7.1	Encoder Decoder Nippu	16
3.7.2	Tarkkaavaisuus (Attention) ja Skaalattu pistetulo tarkkaavaisuus (Scaled Dot-Product Attention)	17
3.7.3	Monihaarainen tarkkaavaisuus (Multi-Head Attention)	17
3.7.4	Asennon Suhteen Eteenpäin Syöttävä Verkko (Position-wise Feed-Forward Networks)	18
3.7.5	Upottaminen ja Softmax	19
3.7.6	Paikallinen encoodaus (Positional Encoding)	19
3.8	Generatiivinen malli	19
3.9	Monitehtäväoppiminen (MTL)	20
3.9.1	MTL arkkitehtuuri	20
4	Apple Inc. osake data	23
4.1	Tekniset indikaattorit	23
4.2	Muuttujien valinta	27
4.3	Datan esikäsittely	28
4.4	Mallien koulutus ja testaus	29
5	Mallien sovittaminen Applen osake dataan	31
5.1	Tulosten vertailua	32
6	Pohdinta	40
6.1	Tulosten yhteenveto ja jatkotutkimus ehdotukset	40
6.2	Luotettavuus ja eettisyys	41
	Lähteet	44

1 Johdanto

Pörssidata on rakenteeltaan aikasarjadataa. Aikasarjadataassa mittaukset on suoritettu järjestyksessä ajan suhteen. Aikasarjadata voidaan ajatella myös järjestysdatana, missä jokaisella mittauksella on oma paikka järjestyksessä. Aikasarjadatoja voi olla muitakin, esimerkiksi teksti voidaan ajatella eräänlaisena aikasarjadataana, missä sanat ovat järjestyksessä tekstin alusta loppuun.

Tarkastellessani aikaisemmissa tutkimuksissa käytettyjä dataja havaitsin että, yleisimmin pörssidatasta mitataan 5 perusattribuuttia: avaushinta, sulkemishinta, korkein hinta, matalin hinta ja volyymi. Tästä esimerkkinä (Patel ym., 2015). Myös tämän työn aineistossa on mitattu samat 5 attribuuttia. Nämä attribuutit voidaan mitata eri väleillä, kuten esimerkiksi tunnin välein tai pörssin aukioloaikana päivittäin. Näitä attribuutteja harvoin käytetään mallintamiseen sellaisinaan. Yleensä mallintamisessa avuksi otetaan niin kutsuttuja teknisiä indikaattoreita. Tekniset indikaattorit ovat laskennallisia mittareita, joita käytetään osake- ja kryptomarkkinoilla analysoimaan hintaliikettä, volyymia ja muita kaupankäynnin tietoja. Monesti vain avaushintaa ja raakadataa käyttäessä voidaan ohittaa jotain tärkeitä informaatiota. Tekniset indikaattorit tarjoavat muun muassa paremman trendin tunnistuksen kuin vain raaka data.(Pring, 2002) (Murphy, 1999)

Pörssidatan ennustamiseen voidaan käyttää joko osakkeen hintaa itsessään tai niin sanottua palautusarvoa tai logaritmista palautusarvoa (eng. return, logarithmic return). Useimmiten palautusarvoa on pidetty parempana mallinnus kohteena, sillä sen arvot ovat normalisoitu paremmin verrattuna itse hintaan. Lisäksi logaritminen palautus kuvaa suhteellista hinnan muutosta, joka on relevantimpaa sijoittajille ja riskin hallitsijoille, sillä se kuvaa sijoituksen kasvua tai laskua. Lisäksi data on näin stationaarista, joka on tarpeellista perinteisten tilastollisten mallien näkökulmasta. Molemmat palautus arvot saattavat antaa yllättävän erilaisia tuloksia, joten tämä on hyvä pitää mielessä malleja tehdessä (Hudson & Gregoriou, 2015).

Perinteisesti aikasarjadataan mallintaminen on suoritettu Autoregressive Integrated Moving Average (ARIMA) malleilla. Nämä mallit toimivat hyvin ja löytävät lineaarisen trendin datan pohjalta. Ne käsittelevät kaikkea malliin jäljelle jäävää virhettä riippumattomana hälynä (eng iid. noise), jota ei voida mallintaa näillä keinoilla. Malleilla löydetään suuntaa antavaa käytöstä esimerkiksi, onko trendi ylös vai alas. Kuitenkin voidaan olettaa, että myös tässä hälyssä saattaa olla jotain säännönmukaisuutta, jota ARIMA malli ei kykene löytämään. (Shumway ym., 2017) Koneoppi-mallit kuten tukivektori regressio, Pitkäkestoinen-lyhytkestomuisti-rakenne (LSTM) ja Transformer saattavat kuitenkin onnistua löytämään jotain piilevää ominaisuutta tästä hälystä.

Toisin kuin perinteiset tilastolliset menetelmät, koneoppimismenetelmät onnistuvat vangitsemaan lineaarisen riippuvuuden lisäksi epälineaarista ja kompleksista riippuvuutta muuttujien väliltä (Ryll & Seidens, 2019). Koneoppimismenetelmien on myös huomattu monesti saavuttavan paremman tarkkuuden datan ennustamisessa kuin perinteiset tilastolliset mallit ja se soveltuu myös epästationaarisen datan mallintamiseen (Khan ym., 2023).

Koneoppimiseen liittyy yhtenä osa-alueena monitehtäväoppiminen (eng. multi-task learning). Siinä voidaan mallintaa kahta tai useampaa tehtävää saman aikaisesti yhden sijasta. Etuja monitehtäväoppimisella on muunmuassa se, että koulutukseen ei tarvitse käyttää kaksinkertaista aikaa, vaan kaksi tehtävää voidaan kouluttaa samanaikaisesti. Koulutuksen tehostamisen lisäksi Park ym. (2022) huomasivat soveltaessaan monitehtäväoppimista pörssi dataan, että mallin tarkkuus parani täydentävän tehtävän avulla verrattuna vain yhden tehtävän malliin.

Tässä työssä tutkitaan koneoppimenetelmien toimintaa pörssikurssien mallintamisessa. Työn tarkoituksena on tarkastella eri koneoppimenetelmien toimintaa ja verrata monitehtäväoppimista yhden tehtävän oppimiseen. Tutkimuskysymykset ovat:

1. Miten monitehtävämallit pärjäävät vertailussa yhden tehtävän mallien ja vertailukohtamallin kanssa?
2. Miten monitehtäväoppiminen vaikuttaa mallien kouluttamiseen?

Malleja on sovellettu vuosilta 1984—2024 kerättyyn pörssiaineistoon. Aineisto sisältää päivittäin mitattua dataa Applen osakkeen hinnasta pörssinavautuessa, korkein hinta päivän aikana, matalin hinta päivän aikana, hinta pörssinsulkeutuessa ja osakkeen vaihdon volyymi.

2 Kirjallisuuskatsaus

Kumar ym. (2021) ovat jakaneet pörssimallintamisen neljään luokkaan, perusanalyysiin, tekniseen analyysiin, perinteiseen tilastolliseen analyysiin ja pehmeään laskentaan. Perinetisiin tilastollisiin malleihin lukeutuu muun muassa ARIMA mallit ja pehmeän laskennan malleihin LSTM ja Transformer. Tässä artikkelissa vertailtiin malleja keskenään, vertailtavia malleja oli muun muassa ARIMA, MARS, Random Forest, RNN ja LSTM. Syvän oppimisen mallit pärjäävät paremmin aikasarjallisen datan käsittelemisessä kuin tilastolliset mallit kuten ARIMA, mutta niiden koulutus on huomattavasti kalliimpaa (Bousqaoui ym., 2021). Koska tämän työn tarkoituksena on löytää mahdollisimman hyvät mallit vertailtavaksi, valittiin menetelmiksi koneoppimisen ja syväoppimisen menetelmiä.

LSTM mallit ovat hyvin laajalti käytettyjä pörssidatan mallintamiseen. Monissa lähteissä kuten (Li ym., 2024) ja (Zeyer ym., 2019) on Transformer mallien huomattu antavan parempia tuloksia ja koulutuksen olevan tehokkaampaa kuin LSTM mallilla. Tästä syystä se on valittu tässä työssä mukaan mallien ja monitehtäväoppimisen (multitask) asetelman vertailuun.

Park ym. (2022) huomasivat LSTM-Forest mallinsa kanssa monitehtäväoppimisasetelman parantavan pörssidatan mallinnuksen tuloksia. Esitelty malli on melko monimutkainen ja vaatii paljon laskentatehoa. Tässä työssä tutkitaan monitehtäväasetelmaa yksinkertaisempien mallien avulla. Tämän työn mallit perustuvat Park ym. (2022) esittelemään malliin ilman Random Forest osaa. Koska mallit ovat näin yksinkertaisempia, on ne mahdollisia laskea yleisesti saatavilla olevilla tietokoneilla. Työssä käytetyt mallit myös ennustavat useampaa kuin yhtä seuraavaa päivää toisin kuin mallit joita Park ym. (2022) käyttivät.

3 Työssä käytetyt menetelmät

Koneoppimisella tarkoitetaan algoritmeja, jotka oppivat kokemuksesta eivätkä ne ole ohjelmoitu tekemään jotain tiettyä tehtävää (Cohen, 2021).

Koulutusalgoritmit pyrkivät muodostamaan parhaan mahdollisen mallin koulutusdatan perusteella. Malli niin sanotusti sovitetaan koulutus dataan. Mallintamisessa on kuitenkin vaarana, että malli kuvaa kyseistä dataa liian hyvin. Tätä kutsutaan ylisovittamiseksi (engl. Overfitting). Tällaisessa tapauksessa koulutettu malli kuvaa hyvin tai lähes kokonaan koulutusdatan käyttäytymistä, mutta se ei kykene yleistämään oppimaansa uudelle tuntemattomalle datalle, vaikka se kuvaisikin samaa ilmiötä (Vartiainen ym., 2021). Mallit pyritään muodostamaan niin, että ne kuvaivat myös uutta dataa mahdollisimman hyvin. Siksi koneoppimisessa käytetäänkin koulutusdatan lisäksi niin sanottua testausdataa, jolla tarkastetaan mallin sopivuus uuden datan kanssa.

3.1 Koulutus ja testaus

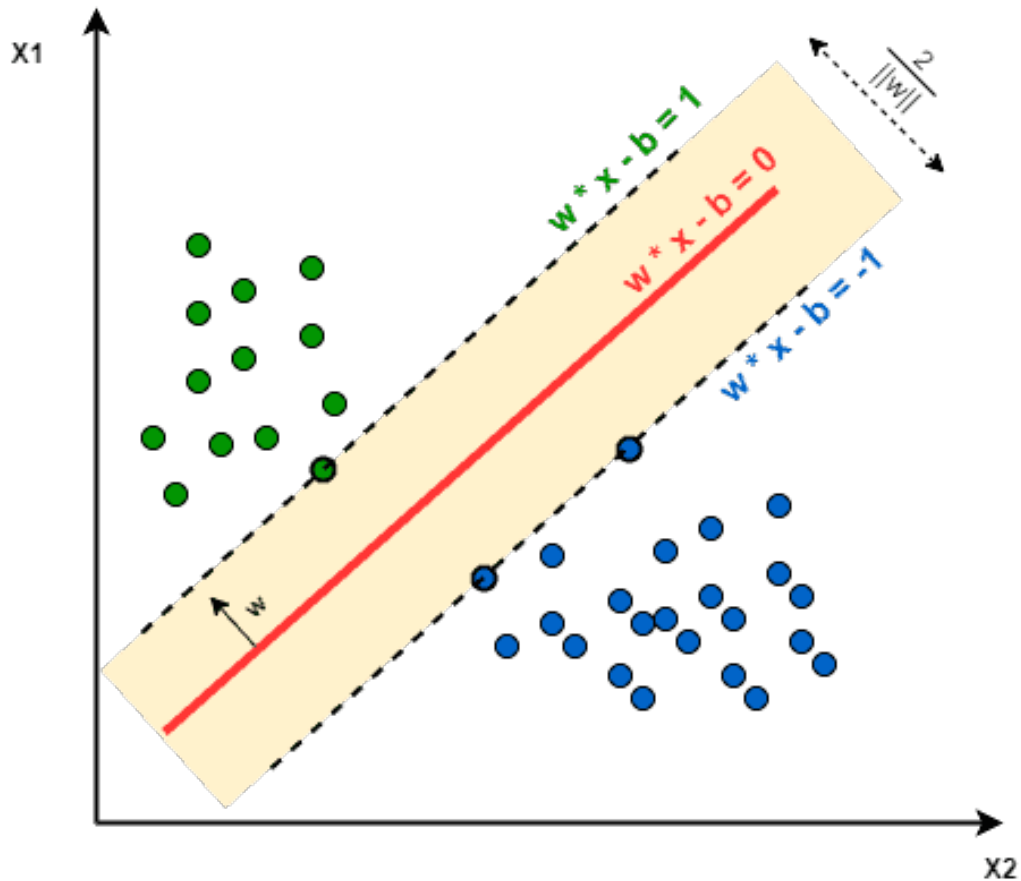
Data jaetaan kahteen osaan joista toista käytetään algoritmin kouluttamiseen (ns. koulutusdata) ja toista taas mallin testaamiseen (ns. testidata). Näin voidaan algoritmia testata samalla datalla kuitenkin siten, että testauksessa käytetään algoritmille tuntematonta dataa. Testaus antaa tietoa siitä, kuinka hyvin algoritmi pystyy yleistämään oppimaansa. Usein mallia kouluttaessa käytetään vielä validointia, jossa koulutusdatasta erotellaan validointi osa hyperparametrien hyvyyden tarkastelemiseksi. Validointi voidaan myös toteuttaa ristiin validoinnilla, jossa validointiosaa poimitaan useamman kerran koulutusdatan erillisestä alijoukosta. Artikkelissaan Sugali ym. (2024) ovat kertoneet eri tapoja datan jakamiseen ja siihen liittyvistä haasteista.

Koneoppimisessa oppiminen voidaan jakaa kahteen luokkaan ohjattuun ja ohjaamattomaan oppimiseen. Ohjattu oppiminen (engl. Supervised Learning) tarkoittaa menetelmää, jossa kohdearvot tai luokat ovat valmiiksi tiedossa. Koska vastemuuttujan todelliset arvot ovat tiedossa, voidaan mallin antamia tuloksia verrata suoraan niihin. Tuloksia vertaamalla saadaan tietoa mallin tarkkuudesta ja voidaan optimoida sitä verrattuna todellisiin arvoihin. Ohjatun oppimisen päätehtävät ovat luokittelu ja regressio. Luokitellussa malli pyrkii jakamaan mallin kahteen tai useampaan luokkaan ja regressiossa taas mallinnetaan lukuarvoa. Kouluttaessa malli pyrkii minimoimaan sopivaa virhemittaria kuten luokitteluvirhettä tai regression neliövirhettä. Tässä työssä käytetään ohjatun oppimisen menetelmiä.

Ohjaamaton oppiminen puolestaan tarkoittaa menetelmiä, joilla ei ole valmiiksi tiedossa olevaa vastemuuttujaa. Esimerkiksi klusterointi on ohjaamaton menetelmä, sillä klustereita ei ole ennalta määrätty. Klusterien lukumäärää arvioidaan erilaisten tilastollisten menetelmien avulla (Mirkin, 2011).

3.2 Tukivektorikone (SVM)

Tukivektorikonetta (engl. support-vector machine) paljon käytetty menetelmä pörsidatan mallintamisessa. Se on ohjatun oppimisen menetelmä, eli vastemuuttujat ovat saatavilla. Perinteinen tukivektorikone pyrkii jakamaan dataa eri luokkiin. Se suorittaa niin sanottua luokittelutehtävää. Data jaetaan hypertasoilla esimerkiksi kahteen luokkaan siten, että marginaalit ovat maksimaaliset ja marginaalien välille ei osu pisteitä kummastakaan luokasta. (Cortes, 1995)



Kuva 3.1. Luokittelu kahteen luokkaan Tukivektorikoneen avulla. Kuvan lähde Chirag (2024)

3.3 Tukivektoriregressio (SVR)

Tukivektoriregressio (engl. Support Vector Regression) on tukivektorikoneen versio, joka soveltuu regressio-ongelmiin. Regressio ongelma on yleistys luokitusongelmasta, jossa malli palauttaa jatkuvan arvon sen sijaan että arvo olisi rajattu tietyille arvoille. SVM:n generalisaatio SVR saadaan aikaan muodostamalla ϵ -putki funktion ympärille. Nyt arvot, jotka osuvat putken sisäpuolelle, eivät aiheuta virhettä mallille. Optimointiongelmana on löytää putki, joka parhaiten arvioi jatkuva-arvoista funktiota samalla tasapainottaen mallin kompleksisuutta ja ennustuksen virhettä. (Awad

ym., 2015) SVR malli voidaan kirjoittaa muotoon:

$$(3.1) \quad f(x) = \begin{bmatrix} w \\ b \end{bmatrix}^T \begin{bmatrix} x \\ 1 \end{bmatrix} = w^T x + b,$$

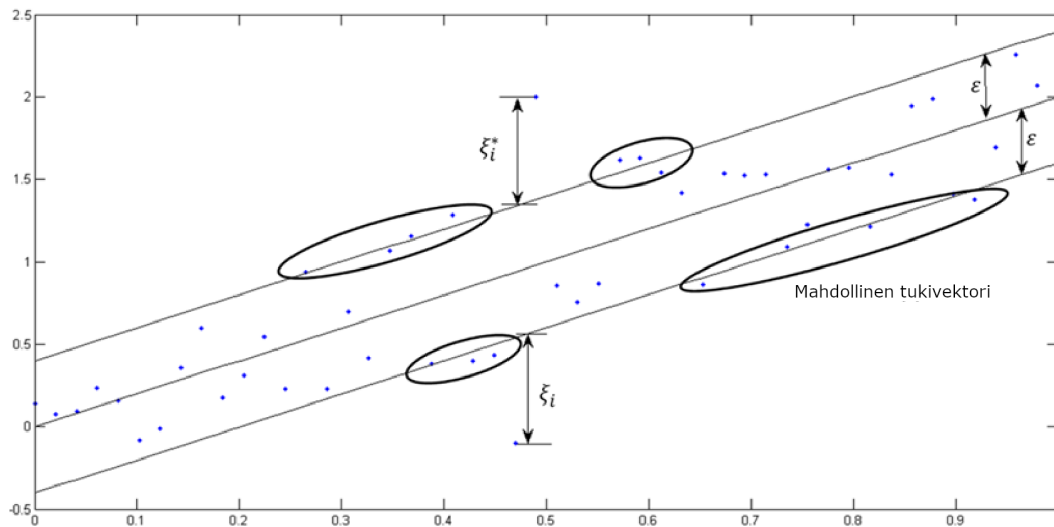
$$x, w \in \mathbb{R}^{M+1}$$

(Awad ym., 2015).

SVR yrittää löytää kapeimman mahdollisen putken hyper- tason ympärille, samalla minimoiden ennustuksen virhettä. Virheellä tarkoitetaan etäisyyttä ennustetun ja todellisen arvon välillä. Edellä mainittu ehto antaa funktion:

$$(3.2) \quad \min_w \frac{1}{2} \|w\|^2,$$

missä $\|w\|$ on normaalivektorin suuruus pinnalle jota approximoidaan (Awad ym., 2015).



Kuva 3.2. Yksiulotteinen tukivektori regressio. Kuvan lähde: (Awad ym., 2015)

Tukivektorkone on hyvä malli, sillä se jättää oudokit eli merkittävästi poikkeavat arvot huomioimatta ja keskittyy ainoastaan sellaiseen aineistoon, joka koetaan olevan mallille oleellisia. SVR:n yksi vahvuuksista on, että laskennallinen kompleksisuus ei riipu syöteavaruuden dimensiosta. Tämä johtuu siitä, että laskenta perustuu $\langle w, x \rangle$ arvoihin ja ne riittävät, kun sisätulo on kerran laskettu. Sisätulo taas voidaan korvata sisätulofunktiolla, joka voi vastata korkeaulotteisen muunnoksen sisätuloa ilman että tarvitsee laskea muunnoksen tuomia monia ulottuvuuksia suoraan. Sen lisäksi Awad ym. 2015) kuvailee mallin olevan hyvä yleistämään ja sen ennustustarkkuuden olevan hyvä. Tukivektori regressio on myös suhteellisen helppo toteuttaa, ja näistä syistä se antaa hyvän vertailukohdan muille monimutkaisemmille malleille.

3.4 Keinotekoiset neuroverkot (ANN)

Keinotekoisilla neuroverkoilla (engl. Artificial Neural Networks) viitataan laskenta-malliin, joka keskeinen idea on lainattu biologisista neuroverkoista (Dongare ym.,

2012). Osa neuroneista ovat niin sanottuja syöttöneuroneita, jotka syöttävät datan muille neuroneille. Data voi olla biologisessa neuroverkossa esimerkiksi ääni, näkö tai kipu. Näissä tapauksissa data on näiden aistien biologisten reseptorien vaste-signaali. Toiset neuronit ovat ulostulo neuroneita, jotka välittävät esimerkiksi näköhavainnon aivojen muuhun prosessointiin. Suurin osa neuroneista on piilossa tällä välillä, jossa ajattelu ja oppiminen tapahtuu (Schmidhuber, 2022).

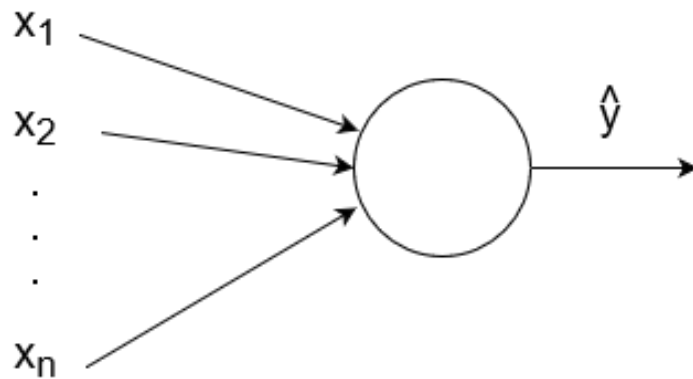
3.4.1 Yhden neuronin perseptron

Perseptroni on varhaisin ja yksinkertaisin neuroverkko malli. Yhden neuronin perseptronille syöte on muotoa,

$$(3.3) \quad net = \sum_{i=1}^{J_1} w_i x_i - \theta = \mathbf{w}^T \mathbf{x} - \theta$$

$$(3.4) \quad y = \phi(net)$$

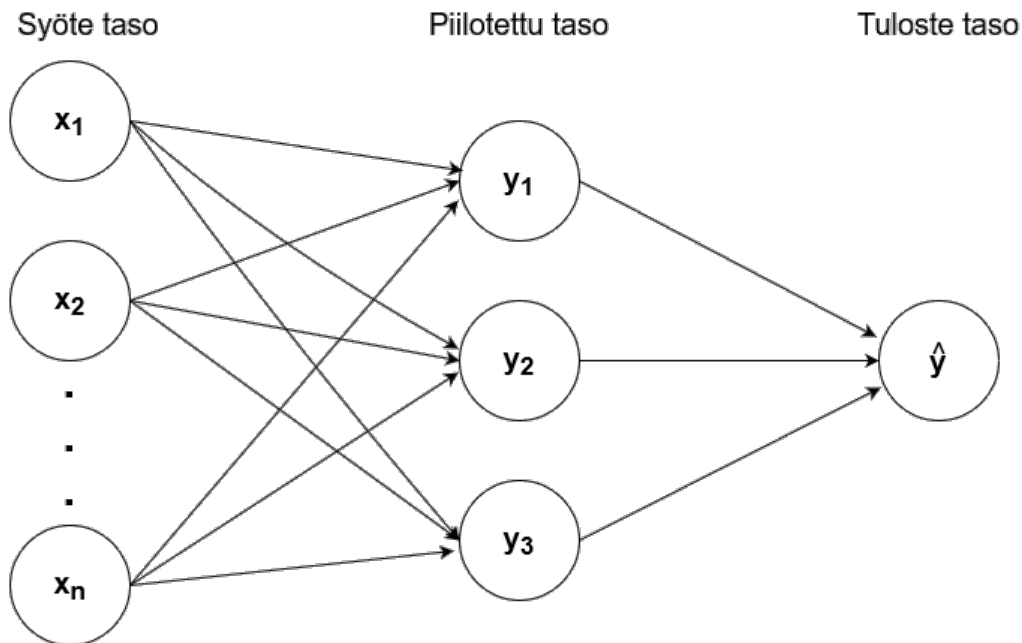
missä x_i on i :nnes syöte w_i on paino parametri i :nne syötteelle. $\mathbf{w} = (w_1, \dots, w_{J_1})^T$, $\mathbf{x} = (x_1, \dots, x_{J_1})^T$, θ on bias termi ja J_1 on syötteiden lukumäärä. Aktivointifunktio $\phi(\cdot)$ on usein jokin jatkuva tai ei-jatkuva funktio, joka heijastaa reaaliarvot välille $(-1,1)$ tai $(0,1)$ (Du & Swamy, 2013). Regressiotehtävässä aktivointifunktio jätetään pois, jotta saadaan ulostulo olemaan rajattomalla välillä. Perseptroneja voi myös olla monta samassa kerroksessa, jolloin esimerkiksi luokittelu useampaan luokkaan onnistuu.



Kuva 3.3. Yhden perseptronin malli

3.4.2 Moni kerroksinen perseptroni

Jos useamman neuronin perseptronista halutaan saada regressioarvo, on verkkoon lisättävä yksi kerros, jossa edellisen kerroksen tulokset kootaan samalla tavalla kuin yhden neuronin perseptronissa. Seuraavan kerroksen syötteet ovat edellisen kerroksen tulosteita kuten kuvassa 3.4. Näin saadaan mallilla laskettua enemmän piileviä tekijöitä kuin yhden neuronin perseptronilla.



Kuva 3.4. Moni kerroksinen perseptron, jossa on yksi piilotettu taso ja tuloste taso. Piilotetulla tasolla on kolme neuronua ja tulostetasolla yksi.

3.4.3 Vastavirta-algoritmi

Neuroverkko koulutetaan ohjatun oppimisen menetelmällä. Siinä neuroverkolle annetaan syötteet ja niitä vastaavat vastemuuttujat. Esimerkkinä tästä on kuva rakennuksesta ja rakennuksen ikkunoiden lukumäärä, jossa kuva on syöte ja ikkunoiden lukumäärä on vaste. (Dongare ym., 2012) Koulutukseen käytettävää algoritmia kutsutaan vastavirta-algoritmiksi (eng. backpropagation).

Vastavirta algoritmi laskee uudet painokertoimet verkon painokertoimille w_{ki}^n , missä n on kerroksen indeksi, k kerroksessa olevan neuronin indeksi ja i kerrokseen tulevan syötteen indeksi. Algoritmi yhden piilotetun tason mallille koostuu neljästä askeleesta:

1. Eteenpäin syöttö
2. Taaksepäin syöttö tulostetasolle
3. Taaksepäin syöttö piilotetulle tasolle
4. Painokertoimien päivitys

(Cilimkovic, 2015)

Eteenpäin syöttö

Ensimmäiseksi lasketaan arvot piilotetuilta tasoilta, joita käytetään vastearvojen laskentaan (Cilimkovic, 2015). Tuominen (2020) esittelee tekstissään kaavat, joilla

vastavirta-algoritmi saadaan laskettua. Eri tasojen tulosteet saadaan:

$$(3.5) \quad \begin{aligned} \phi(w_{k0}^{piilo} + \sum_{i=1}^I (w_{ki}^{piilo} x_i)) &= y_k \\ w_0^{tuloste} + \sum_{k=1}^K (w_k^{tuloste} y_k) &= \hat{y} \end{aligned}$$

Missä $i \in I$ on syöte tason syöte ja $k \in K$ on piilotetun tason neuronin ks. kuva 3.4. w_k^{piilo} kuvaa piilotetun tason neuronin k painokertoimia, $w^{tuloste}$ kuvaa tulostetason painokertoimia ja ϕ epälineaarista aktivointifunktiota.

Taaksepäin syöttö tuloste tasolle

Taaksepäin syöttö alkaa laskemalla tappio funktio käyttämällä todellisia arvoja ja mallin tuloste arvoja. Tappiofunktioilla on pääsy todellisiin arvoihin, eli niin sanottuun Pohjatotuuteen (eng. Ground truth). Yleisimmin käytetty tappiofunktio on keskimääräinen neliövirhe MSE loss (eng. Mean squared error loss).

$$(3.6) \quad \mathcal{L}_{MSE} = \sum_{i=1}^n (y - \hat{y})^2,$$

missä y on mallin tulosteita $\hat{y}$ vastaavat pohja totuudet. Taaksepäin syöttäessä otetaan derivaatta tappiofunktioista $\mathcal{L}_{MSE}$ muuttujan $\hat{y}$ suhteen.

$$(3.7) \quad \frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}}$$

Osittaisderivaatan tulos syötetään taaksepäin tuloste tasolle.

Taaksepäin syöttö piilotetulle tasolle

Tuloste tasolla eteenpäin syöttö on samaa muotoa kuin kohdassa 3.5. Taaksepäin syöttö tapahtuu osittaisderivaattana tappiofunktioista piilotetun kerroksen tulosteiden suhteen.

$$(3.8) \quad \frac{\partial \mathcal{L}_{MSE}}{\partial y_i} = \frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial y_i},$$

missä $\frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}}$ saadaan taaksepäin syöttönä seuraavalta tasolta.

Painokertoimien päivitys

Kuvassa 3.4 on kaksi tasoa, joissa voidaan päivittää mallin painokertoimia eli parametrejä. Painokertoimien päivitykseen käytetään päivityssääntöä:

$$(3.9) \quad \begin{aligned} w_{piilo}^{t+1} &= w_{piilo}^t + \mu \frac{\partial \mathcal{L}_{MSE}}{\partial w_i^{piilo}} = w_{piilo}^t + \mu \frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_i^{piilo}}, \\ w_{tuloste}^{t+1} &= w_{tuloste}^t + \mu \frac{\partial \mathcal{L}_{MSE}}{\partial w_i^{tuloste}} = w_{tuloste}^t + \mu \frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial y_i} \frac{\partial y_i}{\partial w_i^{tuloste}} \end{aligned}$$

missä μ on oppimisnopeus ja w_{piilo}^t ja w_{tulo}^t ovat tasojen painokertoimia epookilla t . Algoritmia toistetaan, joko ennalta määrätty epookkien lukumäärä, tai kunnes tappio ei enää laske merkittävästi. (Tuominen, 2020)

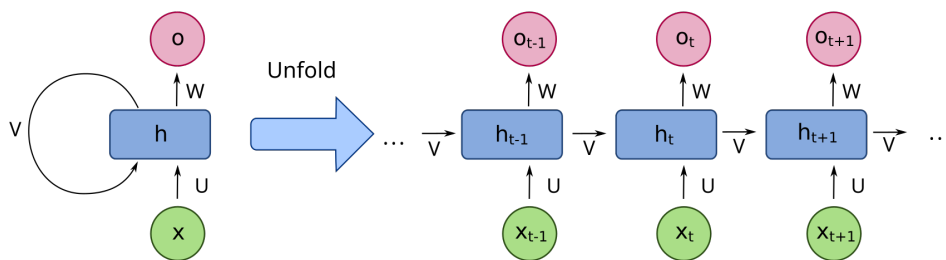
Samaa algoritmia voi soveltaa monimutkaisempiinkin malleihin. Tällaisia voivat olla esimerkiksi mallit, joissa on enemmän neuroneita tasoilla ja enemmän tasoja. Mitä syvemmäksi neuroverkko kasvaa, sitä monimutkaisemmaksi malli muuttuu ja enemmän parametreja malliin syntyy. Mitä enemmän parametreja mallissa on sitä helpommin se ylisovittuu ja mitä enemmän parametreja malli sisältää sitä enemmän dataa tarvitaan sen kouluttamiseen. Neuroverkon syvyys pyritään asettamaan aina mahdollisimman matalaksi kuitenkin siten, että se selittää aineiston riittävästi.

3.5 Takaisinkytketty neuroverkko (RNN)

Takaisinkytketyt neuroverkot (eng. Recurrent Neural Networks) saivat alkunsa 1990-luvulla. Ne on suunniteltu oppimaan sekvenssejä ja ajasta riippuvaisia rakenteita. Niitä on käytetty muun muassa järjestysdataan kuten kirjaimiin sanoissa. (Medsker, Jain ym., 2001)

Takaisin kytketyssä neuroverkossa syöte koostuu vektoreiden sekvensseistä ajan t suhteen esimerkiksi $\{\dots, \mathbf{x}_{t-1}, \mathbf{x}_t, \mathbf{x}_{t+1}, \dots\}$, jossa $\mathbf{x}_t = (x_1, x_2, \dots, x_n)$. Syöte ja vaste parit ovat sekvenssoitu ajan suhteen, mikä tarkoittaa, että syöte muuttujaa $\mathbf{x}_t$ vastaavan vastemuuttujan $\mathbf{y}_t$ pohja totuus voi olla esimerkiksi $\mathbf{x}_{t+1}$. Syötteet voivat olla myös mallin generoimia, siten että seuraavalla kierroksella syötteinä käytetään edellisen kierroksen ulostuloarvoja. Takasin kytketyn neuroverkon kouluttaminen tapahtuu samalla tavalla vastavirta algoritmilla tavallisen keinotekoisien neuroverkon tapaan. (Salehinejad ym., 2017)

Takaisin kytkettyjä neuroverkkoja voidaan käyttää myös generatiivisina malleina, eli mallintamaan vielä tapahtumatonta. Toisin sanoen mallintamaan tulevaisuutta. Tästä lisää luvussa 3.8.



Kuva 3.5. Esimerkki takaisin kytketystä neuroverkosta, jossa on yksi piilotettu taso. Kuvassa o kuvaa mallin tulostetta.

3.6 Pitkäkestoinen-lyhytkestomuisti-rakenne (LSTM)

Pitkäkestoinen-lyhytkestomuisti-rakenne (eng. Long Short Term Memory) on takaisin kytketyn neuroverkon sovellus. Se koostuu takaisin kytketyistä alaverkoista, joita kutsutaan muistiblokeiksi. LSTM-verkko on sama kuin standardi RNN-verkko, paitsi että RNN-verkon solmu (neuroni) on korvattu muistiblokilla ks. kuva 3.6. LSTM-blokit voidaan sekoittaa tavallisten solmun kanssa, mutta yleensä se ei ole tarpeen. LSTM-verkot voivat käyttää samoja vaste tasoja kuin RNN-verkot. (Graves & Graves, 2012)

Ongelmana RNN-verkoilla on, että ne käsittelevät dataa saman arvoisesti. Alkuperäisen datalla ei välttämättä ole samanlaista riippuvuutta, kuin verrattuna esimerkiksi edelliseen arvoon x_{t-1} . Jotain turhaa tai vanhentunutta tietoa pitää unohtaa.

LSTM käyttää hyväkseen niin sanottuja solutilaa ja piilotettua tilaa. LSTM-blokille tulee syötteenä edellisen tason solutila C_{t-1} ja piilotettu tila h_{t-1} . Lisäksi syötteenä annetaan myös data X_t . Kuvassa 3.6 σ tarkoitetaan loogista funktiota, $\tanh$ tarkoittaa hyperbolista tangentialifunktiota, $+$ tarkoittaa summainta ja $\times$ alkiokohtaista tuloa.

LSTM mallin porteissa säädetään, kuinka paljon tietoa edellisestä tapahtumasta säilytetään seuraavaan tapahtumaan. Unohda portti (eng. forget gate) päättää mitä informaatiota poistetaan tai "unohdetaan". Se saadaan laskettua kaavalla:

$$(3.10) \quad f_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f),$$

missä σ kuvaa loogistista funktiota, $\mathbf{W}_f$ kerroin matriisia datalle $\mathbf{x}_t$ ja $\mathbf{U}_f$ kuvaa kerroinmatriisia edelliselle piilotetulle tilalle $\mathbf{h}_{t-1}$. $\mathbf{b}_f$ on bias-termi. Seuraavaksi päätetään, mitä informaatiota säilytetään. Se tapahtuu syöttö portilla (eng. input gate), joka saadaan kaavalla:

$$(3.11) \quad i_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i),$$

missä samalla tavalla kuin edellä $\mathbf{W}_i$ ja $\mathbf{U}_i$ ovat syöte porttiin liittyviä paino matriiseja ja $\mathbf{b}_i$ on bias termi.

$\hat{C}_t$ kuvaa kandidaatti arvoja jotka voidaan listätä tilaan. Se saadaan kaavalla:

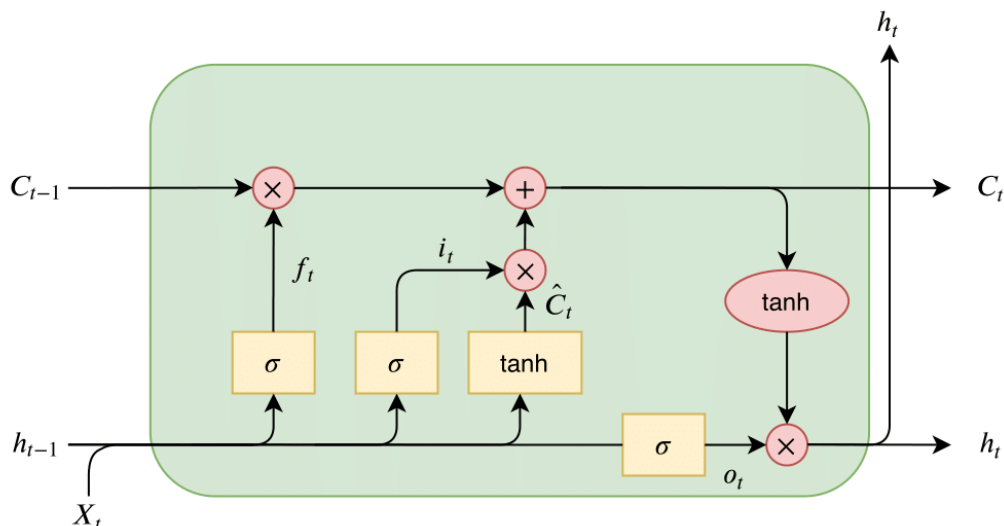
$$(3.12) \quad \hat{C}_t = \tanh(\mathbf{W}_C \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C)$$

Seuraavaksi kerrotaan vanha tila C_{t-1} unohdus portilla f_t , jolloin unohdetaan ne asiat, jotka on päätetty unohtaa. Sen jälkeen siihen lisätään kandidaatti arvot, jotka skaalataan syöte-portin arvoilla.

$$(3.13) \quad C_t = f_t C_{t-1} + i_t \hat{C}_t$$

Nyt on laskettu ajanhetken t solu-tila. Seuraavaksi lasketaan tuloste-portti (eng. output gate) ja uusi piilotettu tila h_t . Tuloste-portti saa informaatiota edelliseltä piilotetulta tilalta ja nykyiseltä datalta:

$$(3.14) \quad o_t = \sigma(\mathbf{W}_o [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

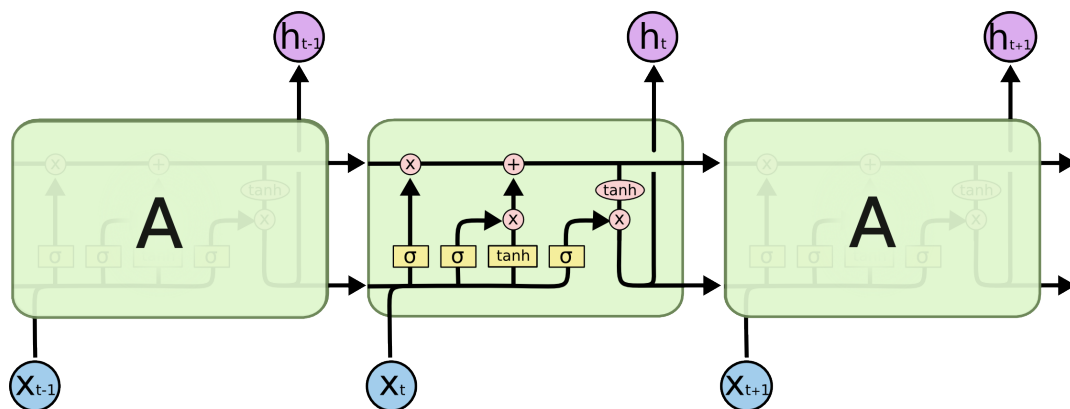


Kuva 3.6. LSTM blokin rakenne

Uusi piilotettu tila saa informaatiota solu-tilalta ja tuloste portilta. Se saadaan kaavalla:

$$(3.15) \quad h_t = o_t \tanh(C_t)$$

LSTM blokit ovat kytkettyinä toisiinsa peräkkäin ja antavat informaatiota eteenpäin ks. kuva 3.7. Jokaisen blokin kohdalla voidaan h_t ajaa tavallisen lineaarisen tason läpi, jolloin se saadaan muutettua halutuksi arvoksi ja voidaan verrata pohja to- tuuteen. LSTM moduuleita on peräkkäin niin monta, kuin on sekvenssointia datassa. Lisää tietoa datan sekvenssoinnista luvussa 4.3. (Lu & Salem, 2017)



Kuva 3.7. Esimerkki kolmesta LSTM modulista

3.7 Transformer

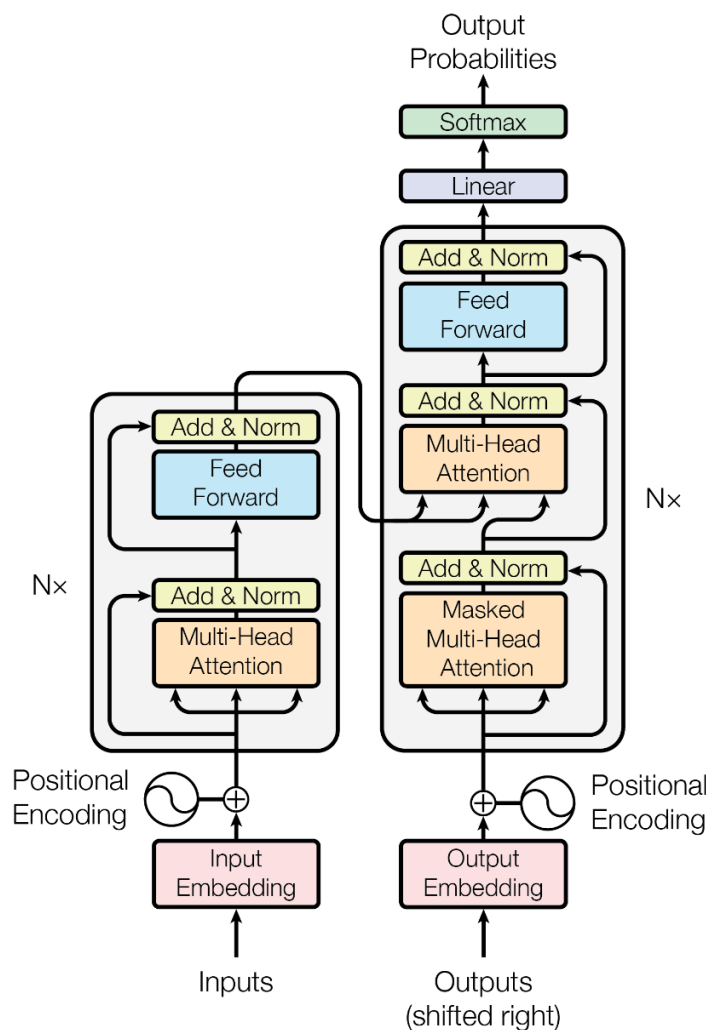
Transformerit esiteltiin ensimmäisen kerran Joulukuussa 2017 seminaari paperissa otsikolla "Attention Is All You Need"(suom. Tarkkaavaisuus on kaikki mitä tarvitset)

(Vaswani, 2017). Tämän paperin kohokohta oli mekanismi nimeltään itsehuomiointi (eng. self-attention).

Transformer arkkitehtuurin idea on siirtyä pois sarjaprosessoinnista (eng. sequential processing), missä syöte annetaan verkolle yksi kerrallaan. Esimerkiksi RNN ja LSTM malleilla syötetään esimerkiksi yksi sana tai aikasarjan tilanteessa päivä kerrallaan. Näissä tilanteissa prosessointi on ainoastaan peräkkäistä. Transformer pyrkii muuttamaan tätä mallia antamalla koko sarjan syötteenä mallille. Tämä mahdollistaa myös rinnakkaisprosessoinnin eri GPU:n ytimillä saman aikaisesti. (Jain, 2022)

3.7.1 Encoder Decoder Nippu

Vaswani (2017) esittelee paperissaan Transformerin arkkitehtuuria. Se koostuu niin sanotusta encoder decoder nipusta.



Kuva 3.8. Transformer arkkitehtuuri jossa encoder ja decoder moduulit.
Kuvan lähde (Vaswani, 2017).

Encoder: koostuu $N = 6$ identtisestä kerroksesta ja jokainen kerros sisältää kaksi alikerrosta. Ensimmäistä alikerrosta kutsutaan monihaaraiseksi itsehuomioin-

timekanismiksi (eng. multi-head self-attention mechanism) ja toinen kerros on yksinkertainen asemakohtainen täysin kytketty eteenpäin syöttävä verkko (eng. Positionwise Fully Connected Feed-Forward Network) (Kuvassa 3.8 **Feed Forward**). Jokaista alikerrosta seuraa residuaalien yhdistäminen jokaisen alikerroksen ohi ja kerros normalisointi (Kuvassa 3.8 **Add & Norm**). Nyt jokaisen alikerroksen tuloste on muotoa:

$$(3.16) \quad \text{KerrosNorm}(x + \text{AliKerros}(x))$$

missä $\text{AliKerros}(x)$ on alikerroksen toteuttama funktio. Residuaaliliitännöjen helpottamiseksi kaikki alikerrokset mallissa kuten myös upotus kerrokset (eng. embedding layers) antavat $d_{\text{malli}} = 512$ dimensioisia tulosteita.

Decooder: koostuu niin ikään $N = 6$ identtisestä kerroksesta. Kahden alikerroksen lisäksi jokaiseen encoder kerrokseen on lisätty kolmas alikerros, joka suorittaa monihaaraisen itsetunnon encoderin tulosteille. Samalla tavalla kuin encoderissa suoritetaan residuaalien yhdistäminen jokaisen alikerroksen ohi, jota seuraa kerros normalisointi. Decooderissa itsehuomiointi alikerrosta muokataan lisäämällä naamiointi kerros (**Mask** kuvassa 3.9), jolla estetään positioita kiinnittämästä huomiota väärin positioihin. Tämä naamiointi yhdistettynä siihen, että tuloste upotukset on siirretty yhdellä oikealle varmistaa sen, että ennuste sijainnille i voi olla riippuvainen vain tulosteista ennen sijaintia i .

3.7.2 Tarkkaavaisuus (Attention) ja Skaalattu pistetulo tarkkaavaisuus (Scaled Dot-Product Attention)

Tarkkaavaisuus funktio voidaan kuvata kyselyn (eng. query) ja avain-arvo –parien kuvauksena tulosteelle, missä kysely, avaimet, arvot ja tulosteet ovat kaikki vektoreita. Tuloste lasketaan painotettuna summana arvoista, missä painot jokaiselle arvolle lasketaan kyselyn yhteensopivuus funktiolla vastaavista avaimista. (Vaswani, 2017)

Vaswani (2017) kutsuu heidän omaa tarkkaavaisuuttaan skaalatuksi pistetulo tarkkaavaisuudeksi (Kuva 3.9 vasemmalla). Tämä tarkkaavaisuus koostuu kyselyistä (**Q** kuvassa 3.9) ja avaimista (**K** kuvassa 3.9) joiden dimensiot ovat d_k ja arvoista (**V** kuvassa 3.9) joiden dimensiot ovat d_v . Kyselyistä lasketaan pistetulo kaikkien avainten kanssa ja jaetaan jokainen arvolla $\sqrt{d_k}$ ja suoritetaan softmax funktio painojen saamiseksi arvoille V . Lopuksi arvot V summataan yhteen näillä painoilla, jotta saadaan kullekin kyselylle sitä vastaava painotettu summa. Katso kaava 3.18.

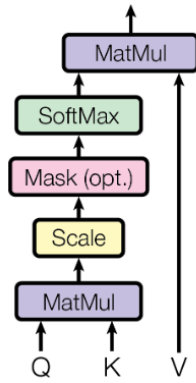
Käytännössä tarkkaavaisuus funktio lasketaan kyselyiden joukolle samanaikaisesti pakattuna samaan matriisiin Q . Myös avaimet ja arvot on pakattu matriiseihin K ja V . Tuloste matriisi lasketaan kaavalla:

$$(3.17) \quad \text{Tarkkaavaisuus}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

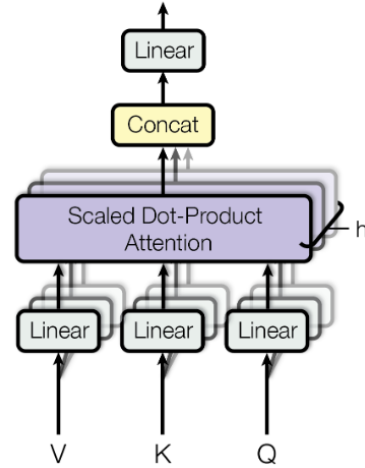
3.7.3 Monihaarainen tarkkaavaisuus (Multi-Head Attention)

Sen sijaan, että suoritettaisiin yksi tarkkaavaisuus funktio d_k dimensioisilla avaimilla, arvoilla ja kyselyillä Vaswani (2017) huomasi lineaaristen projektioiden h kertaa

Scaled Dot-Product Attention



Multi-Head Attention



Kuva 3.9. Tarkkaavaisuus funktiot Transformer -mallissa. Vasemmalla skaalattu pistetulo tarkkaavaisuus ja oikealla monihaarainen tarkkaavaisuus. Kuvan lähde (Vaswani, 2017).

erilaisten opittujen lineaaristen projektoiden kanssa dimensioille d_k , d_q ja d_v olevan hyödyksi. Monihaarainen tarkkaavaisuus on esitetty kuvassa 3.9 oikealla. Jokaiselle näille projisoiduille versioille kyselyistä, avaimista ja arvoista suoritetaan rinnakkain tarkkaavaisuus funktio, josta tulosteena saadaan d_v -dimensioiset tulostearvot. Nämä konkatonoidaan ja vielä kerran projisoidaan johtaen lopullisiin arvoihin kuten kuvasta 3.9 nähdään.

Monihaarainen tarkkaavaisuus mahdollistaa mallin saavan informaatiota eri lailla esiintyvistä aliavaruuksista ja erilaisissa kohdissa. Yhdellä tarkkaavaisuus haaralla keskiarvon laskeminen estää tämän. Minihaarainen tarkkaavaisuus saadaan laske-malla:

$$(3.18) \quad \begin{aligned} \text{MoniHaara}(Q, K, V) &= \text{Concat}(\text{haara}_1, \dots, \text{haara}_h)W^O, \\ \text{haara}_i &= \text{Tarkkaavaisuus}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned}$$

Missä projektiot ovat parametrimatriiseja $W_i^Q \in \mathbb{R}^{d_{\text{malli}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{malli}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{malli}} \times d_v}$ ja $W_i^O \in \mathbb{R}^{hd_v \times d_{\text{malli}}}$.

3.7.4 Asennon Suhteen Eteenpäin Syöttävä Verkko (Position-wise Feed-Forward Networks)

Niin encorder kuin decorderkin sisältävät jokaisen tarkkaavaisuus alikerroksen lisäksi täysin kytketyn eteenpäin syöttävän verkon, joka on suoritettu jokaiselle paikalle erikseen ja samalla tavalla. Tämä koostuu kahdesta lineaarisesta muutoksesta ja ReLU näiden muutosten välissä.

$$(3.19) \quad FNN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Vaikka lineaariset transformaatiot ovat samat eri positioilla, ne käyttävät eri parametreja eri kerroksilla. Tämän voi ajatella myös konvoluutiona, jonka kernelin koko on

1. Syötteen ja tulosteen dimensio on $d_{malli} = 512$, ja sisemmän kerroksen dimensio on $d_{ff} = 2048$. (Vaswani, 2017)

3.7.5 Upottaminen ja Softmax

Transformer -mallissa käytetään opittuja upotuksia, jotta syöte ja tuloste alkioit saadaan d_{malli} dimensioisiksi vektoreiksi. Upotus (eng. embedding) on kuvaus, joka muuttaa kaikki kategoriset indeksit sitä vastaavaksi opituksi vektoriksi. Mallissa käytetään myös opittuja lineaarisia transformaatioita ja softmax funktiota muuttamaan decoderin tuloste ennustamaan seuraavan alkion todennäköisyyttä. Malli jakaa saman paino matriisin upotus kerroksien ja ennen softmaxia toteutettuneiden lineaaristen transformaatioiden välillä. Upotuskerroksessa nuo painot ovat kerrottu luvulla $\sqrt{d_{malli}}$.

3.7.6 Paikallinen encoodaus (Positional Encoding)

Koska malli ei sisällä yhtään takaisin kytkentää eikä konvoluutiota ja jotta datan sekvenssien sisältämää syötteiden järjestystä voidaan käyttää hyväksi, on siihen lisättävä informaatiota alkioiden absoluuttisista paikoista sekvenssissä. Siispä on lisättävä paikallinen encoodaaminen syöte upotuksiin encoder ja decoder nippujen pohjalta. Encoodauksilla on sama dimensio kuin upotuksilla, joten ne voidaan summata yhteen.

Vaswani (2017) on käyttänyt julkaisussaan sini ja cosini funktioita eri taajuuksilla:

$$(3.20) \quad \begin{aligned} PE_{(pos,2i)} &= \sin(pos/10000^{2i/d_{malli}}) \\ PE_{(pos,2i+1)} &= \cos(pos/10000^{2i/d_{malli}}), \end{aligned}$$

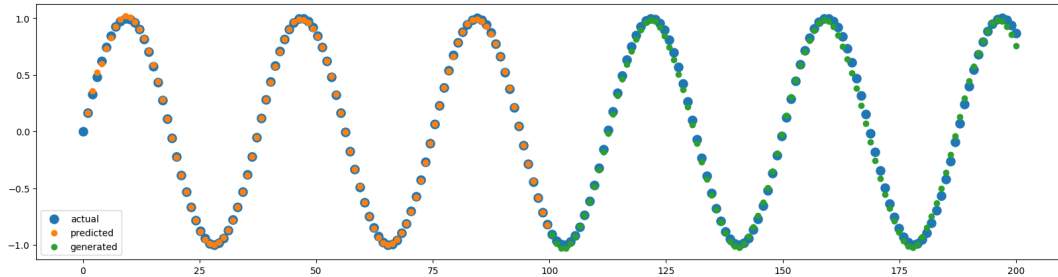
missä pos on positio ja i on dimensio. Arvot lasketaan siis usealle arvolle i siten, että piirteitä on yhtä paljon kuin upotuksilla, ja näin saatu piirrevektori summataan mukaan kyseisen syötteen upotuksen kanssa. Nyt jokainen positio vastaa sinusiodia. Lisää tapoja toteuttaa paikallinen encoodaaminen voi lukea muunmuassa (Chen ym., 2021) ja (Zheng ym., 2021).

3.8 Generatiivinen malli

Generatiivisella mallilla tarkoitetaan mallia, joka pystyy generoimaan uusia näytteitä, jotka muistuttavat sen opetusdataa. Aikasarjaaineistoon generatiivista mallia voidaan soveltaa siten, että tiedettyjen syöte sekvenssien avulla generoidaan tulosteita jotka seuraavat aikaisempaa sekvenssiä. Jos generoidut tulosteet sisällytetään syötteeseen, voidaan generoida loputon määrä tulosteita. Edellisten arvojen ja niiden käytöksen perusteella voidaan tulostaa seuraava arvo. Kuvassa 3.10 on esitetty yksinkertainen sinikäyrä, josta osa pisteistä on sisällytetty harjoitusdataan, joiden avulla on generoitu loput käyrän pisteistä. Generatiivisella mallilla ei ole tietoa generoitavien pisteiden pohjatotuudesta, vaan ne muodostetaan jo koulutetun mallin avulla.

Erilaisia RNN verkkoja on käytetty generatiivisina malleina kuten Jordan RNN, Elman RNN ja LSTM. Myös transformer mallit pystyvät tuottamaan generoituja arvoja, sillä datan järjestys on encoodattu syötteisiin (luku 3.7.6).

Tunnetuin sovellus generatiivisista malleista on varmasti tekstin generoiminen. Erilaisia generatiivisia kuten ChatGPT saatetaan käyttää esimerkiksi tiedon hakuun. Mallit alkavat olemaan jo niin hyviä, että muun muassa oppilaitoksissa on pitänyt ottaa käyttöön menetelmiä generatiivisten mallien käytön tunnistamiseksi. (Laakkonen, 2023) Tässä työssä generatiivisia malleja on käytetty tuottamaan muun muassa seuraavien päivien logaritmisia palautusarvoja. Näistä lisää luvussa 5.



Kuva 3.10. Esimerkki yksinkertaisen generatiivisen mallin tulosteesta. Oranssit pisteet ovat ennustettuja pisteitä, vihreät ovat mallin generoimia pisteitä ja siniset ovat toteutuneita pisteitä.

3.9 Monitehtäväoppiminen (MTL)

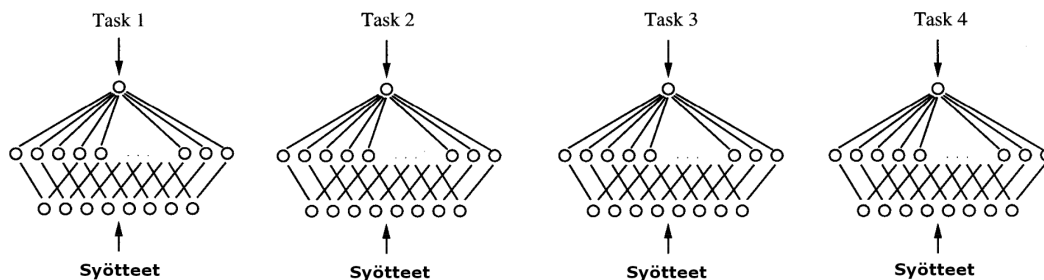
Koneoppimisessa tarvitaan usein suuria määriä nimettyä dataa, jotta saadaan koulutettua hyvä malli. Syvä neuroverkot tuottavat paljon parametreja, joten koulutusdatan pitää olla myös tarpeeksi suuri. Kuitenkin joillain aloilla kuten lääketieteellisessä kuvantamisessa ei ole mahdollista nimetä kaikkia tarvittavia datapisteitä, sillä nimeäminen on hyvin työintensiivistä. Monitehtäväoppiminen auttaa tässä tapauksessa käyttämällä hyödyllistä informaatiota muista toisiinsa liittyvistä oppimistehtävistä. (Zhang & Yang, 2018)

Monitehtäväoppiminen (eng. Multi-task learning) tarkoittaa, että kaksi tai useampi tehtävä opitaan saman aikaisesti tehtävien suhteen yhteisellä mallilla. Tämä tuo uusia optimointi haasteita, kuten mitkä tehtävät pitäisi oppia samanaikaisesti. Tämä on jo sinänsä epätriviaali ongelma. Monitehtävä oppimisen on kuitenkin todettu auttavan mallin tarkkuutta toisiinsa liittyvien tehtävien kohdalla. (Crawshaw, 2020)

3.9.1 MTL arkkitehtuuri

Monitehtäväoppiminen käyttää kaikille tehtäville yhteistä verkkoa tehtävien oppimiseen. Tällaisessa verkossa piilokerrokset ovat samat kaikille tehtäville, mutta ulos- tulokerrokset ovat tehtäväkohtaisia. Otetaan esimerkiksi neljä tehtävää, joita malli pyrkii oppimaan. Yksinkertainen lähestymistapa tälle tilanteelle on, että kaikille tehtäville tehdään omat toisistaan riippumattomat mallit. Kuva 3.11. Tätä kutsutaan yhden tehtävänoppimiseksi (eng. single task learning (STL)). Siinä malli oppii vain oman tehtävänsä ominaisuudet eikä tiedon jakamista tapahdu. (Caruana, 1997)

Monitehtäväoppiminen käyttää hyväkseen kaikkien tehtävien informaatiota. Tässä tapauksessa tehtävät jakavat osan verkkoa, jolloin jokainen tehtävä vaikuttaa osal-



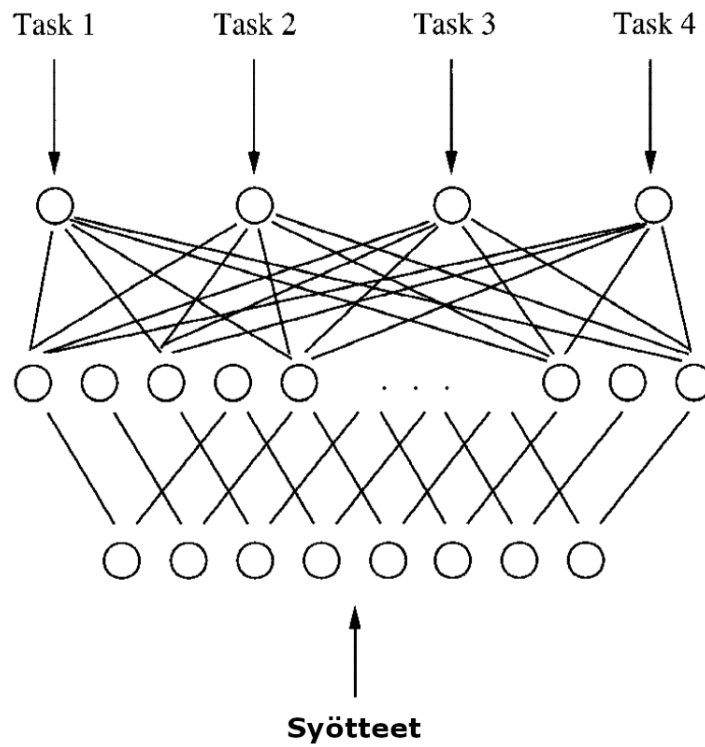
Kuva 3.11. Neljän tehtävän mallit yhden tehtävän oppimisella (Caruana, 1997)

taan siihen, miten verkon koulutus muokkaa parametreja. Kuva 3.12. Monitehtävä oppimisen keskeinen idea on jakaa tietoa siitä, mitä on opittu eri tehtävien toimesta samanaikaisesti tehtävien koulutuksen kanssa. Monitehtäväoppimisessä piirteistä, jotka kehittyvät piilotetuissa kerroksissa yhdelle tehtävälle, voidaan käyttää hyväksi toisten tehtävien toimesta. MTL mahdollistaa joidenkin piilotettujen yksiköiden erikoistumisen yhteen tai pieneen osaan tehtäviä. Muut tehtävät voivat sivuuttaa piilotetut yksiköt, jotka eivät ole niiden kannalta hyödyllisiä pitämällä niiden painokertoimet pieninä. Monissa tapauksissa MTL voidaan hyödyntää, kun halutaan parantaa tarkkuutta yhden pää tehtävien suhteen. MTL voidaan käyttää yleistämisen parantamiseen, oppimisen nopeuttamiseen ja opittujen mallien ymmärrettävyyden parantamiseen. (Caruana, 1997)

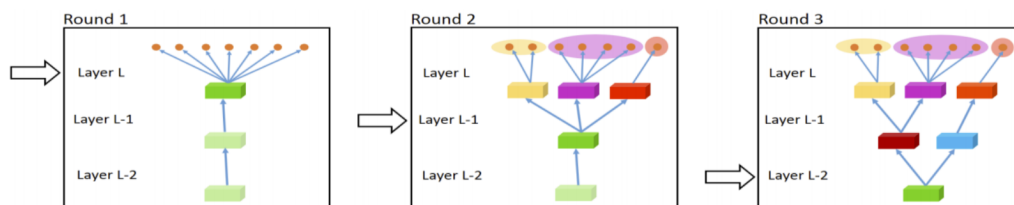
Kuvassa 3.12 jokainen tehtävä käyttää samoja syöte ja piilokerroksia ja niiden parametreja. Myös mallin arkkitehtuuri voidaan asettaa oppimisongelmaksi. Tällöin malli voi oppia miten parametrien pitäisi olla jaettu tehtävien kesken. Samankaltaiset tehtävät jakavat enemmän verkkoa kuin vähemmän toisiinsa liittyvät tehtävät (Crawshaw, 2020).

Jos tehtävät eivät ole kaikki yhtä hyvin yhteydessä toisiinsa voidaan verkon arkkitehtuuria muuttaa menetelmillä kuten haaroitettu jakaminen (eng. branched sharing). Kuvassa 3.13 on esitelty vaiheet, joilla kerrokset jaetaan eri tehtäville niiden samankaltaisuuden mukaan. Koulutuksen edetessä vähemmän yhteydessä olevat tehtävät siirtyvät toisiin clustereihin, jolloin eniten yhteydessä olevat tehtävät jakaisivat mahdollisimman suuren määrän parametreja. (Lu ym., 2017) Tässä työssä ei keskitytä arkkitehtuurin oppimiseen vaan tehtävät on oletettu olevan toisiinsa yhteydessä.

Monitehtäväoppimisen asettelu voidaan myös ajatella siten, että yksi tehtävä otetaan päätehtäväksi (eng. task of intrest) ja seuraavat tehtävät ovat niin sanottuja täydentäviä tehtäviä (eng. supplementarytasks) (Kaski & Peltonen, 2007). Näin voidaan tarkastella täydentävien tehtävien vaikutusta päätehtävän tarkkuuteen. Tässä työssä toteutetaan monitehtäväoppiminen siten, että pyritään vertailemaan tavallisen mallin ja monitehtävämallin eroja ja mahdollisia tulosten parannuksia. Tästä lisää luvussa 5.



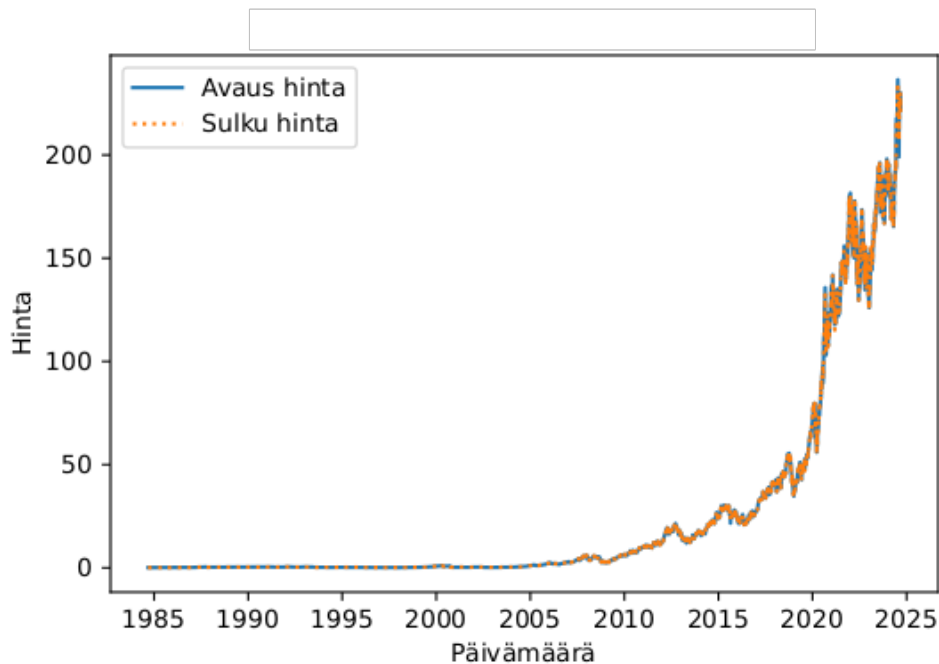
Kuva 3.12. Neljän tehtävän malli monitehtäväoppimisella (Caruana, 1997)



Kuva 3.13. Opittu haarautumisarkkitehtuuri joka esiteltiin (Lu ym., 2017).

4 Apple Inc. osake data

Data on kerätty kaggle sivustolta. Siihen on kirjattu Applen osakkeen hinta pörssin avautuessa, korkein hinta päivän aikana, matalin hinta päivän aikana, hinta pörssin sulkeutuessa ja osakkeen vaihdon volyymi. Aineistossa on päivittäistä dataa vuosilta 1984–2024. ("Apple (AAPL) Stock Dataset 1980-2025 — kaggle.com", ei julkaisupäivää)



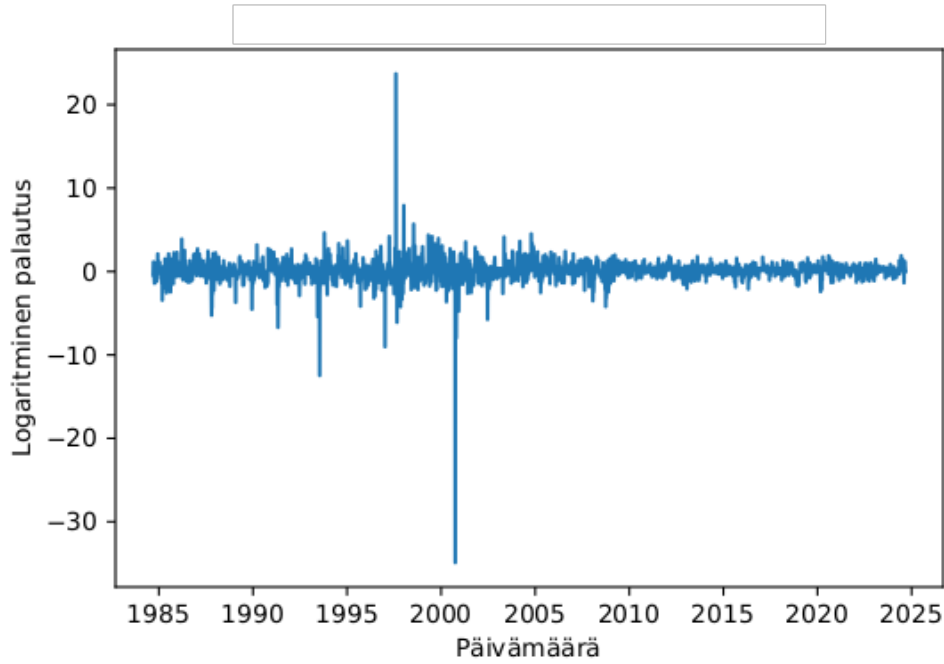
Kuva 4.1. Osakkeen avaus- ja sulkuhinnan kehitys mittausajalta.

4.1 Tekniset indikaattorit

Datasta on laskettu erilaisia indikaattoreita ja tunnusarvoja, jotka helpottavat sen analysointia ja mallinnusta. Päätehtävänä on käytetty logaritmista palautusarvoa (eng. logarithmic return). Se kuvaa miten hinta on käyttäytynyt päivän aikana. Logaritminen palautusarvo saadaan laskettua kaavalla:

$$(4.1) \quad LR = 100 \cdot \log(C_{t-1}/C_t),$$

missä C_t on pörssin sulkuhinta ajanhetkellä t . Logaritminen palautusarvo kuvaa investoinnin tuottoa halutulla aikavälillä (Panna, 2017). Tässä tapauksessa, koska päivät ovat peräkkäiset, arvo kuvaa yhden päivän prosentuaalista muutosta. Positiivinen arvo kertoo arvon nousseen ja negatiivinen sen laskeneen. Mitä kauempana nolasta luku on, sitä enemmän arvo on muuttunut.



Kuva 4.2. Logaritmisen palautuksen kehitys mittausajalta.

Päätehtävän lisäksi aineistosta on laskettu muita indikaattoreita, joita on käytetty mallin koulutuksessa: Chaikin Money Flow, Force Index, Moving Average Convergence Divergence, Vortex Indicator, KST Oscillator, Aroon Indicator, PSAR up trend value indicator, PSAR down trend value indicator, The Percentage Price Oscillator, Percentage Price Oscillator Histogram, The Percentage Volume Oscillator, The Percentage Volume Oscillator Signal Line, The Percentage Volume Oscillator Histogram ja Logarithmic Return. Nämä indikaattorit on todettu vaikuttavan eniten päätehtävän vaihteluun. Indikaattorien valinnasta lisää luvussa 5. Indikaattorit ovat laskettu Pythonin Technical Analysis kirjastolla, joka on avoimen lähdekoodin ohjelma. Indikaattorien kuvaus perustuu heidän kuvauksiinsa kirjaston sivuilla (Padial, 2018).

Chaikin Money Flow (CMF) kuvaa rahan virtausta tietyssä ajan jaksossa. CMF lasketaan kaavalla:

$$(4.2) \quad CMF = \sum_{i=0}^n [((C_i - L_i) - (H_i - C_i)) / (H_i - L_i) * V_i] / \sum_{i=0}^n V_i,$$

missä C on sulkemis hinta, L on alin hinta, H on korkein hinta ja V on vaihto volyymi. Tyypillisesti n on 21 päivän ajanjakso.

Force Index kuvaa kuinka suuri osto tai myynti paine on. Suuret positiiviset arvot vahvaa nousevaa trendiä ja pienet arvot vahvaa laskevaa trendiä. FI saadaan kaavalla:

$$(4.3) \quad FI = (C_t - C_{t-1}) * V,$$

missä C_t on nykyinen sulkemis hinta, C_{t-1} on edellinen sulkemishinta ja V on volyymi.

Moving Average Convergence Divergence on trendiä seuraava momentti indikaattori, joka kuvaa yhteyttä kahden liikkuvan keskiarvon välillä. Se saadaan kaavalla:

$$(4.4) \quad \begin{aligned} MCAD &= MA_{12}(C) - MA_{26}(C), \\ Signaali &= MA_9(MCAD), \\ MCADhist &= MCAD - Signaali, \end{aligned}$$

missä $MA_{12}(C)$ on 12 päivän liikkuva keskiarvo hinnasta, $MA_{26}(C)$ on 26 päivän liikkuva keskiarvo hinnasta, $MA_9(MCAD)$ on 9 päivän liikkuva keskiarvo MCAD:ista.

Vortex Indicator koostuu kahdesta oskillaattorista jotka mittaavat positiivisen ja negatiivisen trendin muutosta. Positiivinen ja negatiivinen trendi lasketaan kaavoilla:

$$(4.5) \quad \begin{aligned} VM^+ &= |H_t - L_{t-1}|, \\ VM^- &= |L_t - H_{t-1}|, \\ VM14^+ &= \sum_k^{k+14} VM^+, \\ VM14^- &= \sum_k^{k+14} VM^-, \end{aligned}$$

missä H_t on korkein arvo päivänä t ja L_t matalin arvo päivänä t . 14 periodin todellinen vaihteluväli saadaan kaavalla:

$$(4.6) \quad \begin{aligned} TR14 &= \sum_{t=k}^{k+14} TR_t, \\ TR &= \max[H_t - L_t, |L_t - H_{t-1}|, |L_t - H_{t-1}|] \end{aligned}$$

Vortex indikaattorit saadaan laskemalla:

$$(4.7) \quad \begin{aligned} VI14^+ &= VM14^+/TR14, \\ VI14^- &= VM14^-/TR14 \end{aligned}$$

KST Oscillator saadaan laskettua neljän erilaisen muutos määrän avulla (eng. rate of change),

$$(4.8) \quad \begin{aligned} ROC_1 &= (C_t/C_{t-10} - 1) * 100, \\ ROC_2 &= (C_t/C_{t-15} - 1) * 100, \\ ROC_3 &= (C_t/C_{t-20} - 1) * 100, \\ ROC_4 &= (C_t/C_{t-30} - 1) * 100, \end{aligned}$$

missä C_t on sulkemishinta päivänä t . Näistä KST Oscillator saadaan laskemalla:

$$(4.9) \quad KST = MA_{10}(ROC_1) * 1 + MA_{10}(ROC_2) * 2 + MA_{10}(ROC_3) * 3 + MA_{15}(ROC_4) * 4$$

Aroon Indicator kertoo milloin trendi on todennäköisesti vaihtamassa suuntaa. Se saadaan laskemalla:

$$(4.10) \quad \begin{aligned} AroonUp &= ((N - DaysSinceNdayHigh)/N) * 100, \\ AroonDown &= ((N - DaysSinceNdayLow)/N) * 100, \\ AroonIndicator &= AroonUp - AroonDown, \end{aligned}$$

missä N on periodin pituus ja $DaysSinceNdayHigh$ on päivien lukumäärä taaksepäin, jolloin N periodin korkein arvo saavutettiin. Mitä pienempi väli on sitä korkeampi $AroonUp$ arvo on. Sama sääntö pätee $AroonDown$ arvon kanssa, mutta $DaysSinceNdayLow$ kuvaa aikaa joka on kulunut N periodin pienimmästä arvosta.

PSAR up trend value indicator

$$(4.11) \quad CSAR = PSAR + PAF(PEP - PSAR)$$

$PSAR$ on edellisen periodin SAR arvo, EP on korkein high arvo nykyisestä ylös trendistä (periodi jolloin hinnan kehitys on ollut nousevaa). AF alkaa .02 ja joka kerta kun ääriarvo tekee uuden suurimman arvon AF kasvaa .02. AF maksimi on .20 riippumatta kuinka pitkälle ylös trendi jatkuu. P tarkoittaa edellistä periodia ja C nykyistä. **PSAR down trend value indicator** lasketaan samalla tavalla, mutta siinä keskitytään alaspäin trendin tarkasteluun.

The Percentage Price Oscillator on momentti oskillaattori, joka mittaa kahden liikkuvan keskiarvon erotusta prosenttina suuremmasta liikkuvasta keskiarvosta. Se saadaan laskettua:

$$(4.12) \quad PPO = [(EMA_{12} - EMA_{26})/EMA_{26}] * 100,$$

missä EMA_n on sulkemishinnan eksponentiaalinen liikkuva keskiarvo periodilla n .

Percentage Price Oscillator Histogram saadaan laskettua singnaali viivan ja PPO:n avulla.

$$(4.13) \quad \begin{aligned} SL &= EMA_9(PPO), \\ PPOHist &= PPO - SL \end{aligned}$$

The Percentage Volume Oscillator mittaa kahden volyymipohjaisen liikkuvan keskiarvon erotusta prosenttiosuutena suuremmasta liikkuvasta keskiarvosta. Se saadaan kaavalla:

$$(4.14) \quad PVO = [(EMA_{12}(V) - EMA_{26}(V))/EMA_{26}(V)] * 100,$$

missä EMA_n on volyymin liikkuva keskiarvo periodilla n .

The Percentage Volume Oscillator Signal Line saadaan 9 päivän eksponentiaalisen liikkuvana keskiarvona PPV:sta seuraavasti:

$$(4.15) \quad SL = EMA_9(PVO)$$

The Percentage Volume Oscillator Histogram saadaan Signaali viivan ja PVO:n erotuksena.

$$(4.16) \quad PVOHist = PVO - SL$$

Koulutukseen käytettyjen indikaattorien lisäksi toiseksi tehtäväksi monitehtävämalleihin on valittu **Average True Range** (ATR) indikaattori. Se on indikaattori hinnan volitaliteetista. Vahvat muutokset kumpaankin suuntaan usein viittaavat suuriin etäisyyksiin. ATR saadaan laskemalla:

$$(4.17) \quad \begin{aligned} ATR &= [(PriorATR * 13) + TR_t]/14, \\ TR_t &= \max(H_t - L_t, |H_t - C_{t-1}|, |L_t - C_{t-1}|) \end{aligned}$$

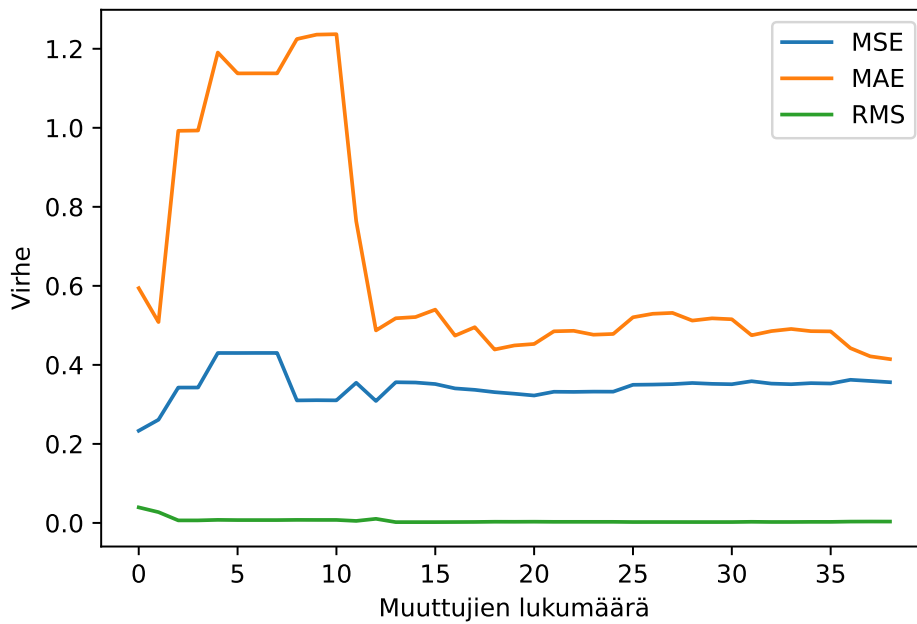
4.2 Muuttujien valinta

Technical analysis kirjastosta löytyy yhteensä 86 erilaista teknistä indikaattoria pörs-sikurssien käytökselle. Jos kaikkia indikaattoreita käytetään yhdessä mitattujen viiden muuttujan kanssa, muuttujia mallille tulee yhteensä 91. Tämä on todennäköisesti liikaa, sillä moni muuttuja saattaa sisältää redundanttia tietoa vastemuuttujasta tai ei tietoa ollenkaan. Jotta saataisiin vain ne muuttujat mukaan, joilla on vaikutusta vastemuuttujaan, pitää muuttujia karsia huomattavasti. Tämä on suoritettu eteenpäin valinnalla.

Eteenpäin valinta (eng. forward selection) suoritetaan siten, että ensiksi malliin valitaan selittävä muuttuja, joka kuvaa parhaiten vastemuuttujaa. Seuraavaksi käydään läpi kaikki mahdolliset jäljellä olevat parit jo malliin valitun muuttujan kanssa. Näistä yhdistelmistä valitaan pari, joka kuvaa vastemuuttujaa parhaiten (Mao, 2004)(Marcano-Cedeño ym., 2010). Tällä tavalla malliin lisätään muuttujia, kunnes kaikki muuttujat on sisällytetty malliin. Jokaisessa kohdassa mallia voidaan arvioida erilaisilla virhe termeillä kuten MSE ja MAPE. Paras malli on se, missä virhe on pienin. Jos monilla malleilla on sama virhe, valitaan malli, jossa on vähiten muuttujia.

Eteenpäin valinta valitsee siis parhaan yhdistelmän muuttujia aloittaen parhaasta yksittäisestä muuttujasta. Tällä menetelmällä ei välttämättä löydetä parasta yhdistelmää, sillä se ei vertaa kaikkia mahdollisia yhdistelmiä. Otetaan esimerkiksi tapaus, jossa muuttujan A selitys tehtävästä T on parempaa kuin muuttujien B ja C . Merkitään $A > B > C$. Eteenpäin valinta valitsee malliin muuttujan A . Voi olla mahdollista, että B ja C muuttujat yhdessä selittävät hyvin tehtävän T . Merkitään nyt $BC > AB > AC$. Koska eteenpäin valinta valitsee parhaalle muuttujalle parhaan mahdollisen vaihtoehdon, se valitsee malliin muuttujat A ja B , vaikka muuttujat B ja C olisi parempi vaihtoehto. Kaikkien yhdistelmien laskeminen olisi kuitenkin liian työlästä ja kallista. Eteenpäin valinta valitsee kuitenkin hyvät vaihtoehdot ja minimoi redundanssia muuttujien välillä.

Kuvissa 4.3 ja 4.4 nähdään, että virhe on alimmillaan 14 muuttujan kohdalla tai se ei enää merkittävästi muutu. Muuttujien määräksi eri malleissa on valittu 14, koska se näyttäisi olevan hyvä määrä eri virhemittojen perusteella. Jotta malleista saataisiin mahdollisimman hyvät, olisi hyvä suorittaa virheen mittausta käyttäen kaikkia käytössä olevia malleja. Jotkin mallit kuten Transformer ovat kuitenkin todella hitaita kouluttaa ja virheiden mittausta veisi turhan paljon aikaa. Virheiden mittausta on suoritettu käyttämällä monikerroksista perseptron verkkoa. Kaikkia 14 muuttujaa käytetään malleissa, joita vertaillaan taulukossa 5.1.



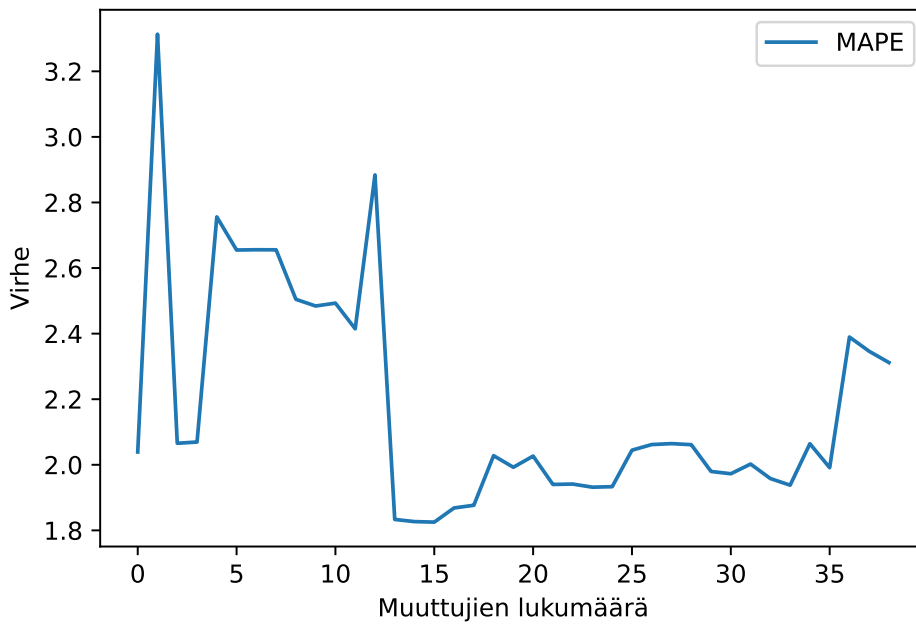
Kuva 4.3. Virheen muutos eri muuttujien lukumäärälle Applen osake datassa.

4.3 Datat esikäsittely

Koska syöte arvot ovat hyvin eri suuruksia, on niiden arvot normalisoitava, jotta yksi arvo ei vaikuttaisi suhteessa muita enemmän malliin. Esimerkiksi muuttuja A saa arvoja välillä $[0, 100]$ ja muuttuja B välillä $[0, 0.2]$ silloin suhteessa pieni muutos muuttujassa A vaikuttaa enemmän kuin suuri muutos muuttujassa B . Normalisoinnissa kaikki muuttujat asetetaan samalle vaihteluvälille. Aineisto on normalisoitu Scikit-learn kirjaston MinMaxScaler funktiolla, joka muuttaa arvot välille $[0, 1]$. Nyt siis jokainen muuttuja saa arvoja välillä 0 ja 1, jolloin kaikkien muuttujien vaihtelu vaikuttaa malliin suhteessa saman verran ja yksi muuttuja ei ole tärkeämpi vain siksi, että se saa suurempia arvoja kuin muut.

Brownlee (2018) käsittelee liukuvan ikkunan menetelmän (eng. sliding-window) soveltamista aikasarja-aineiston muuntamiseen valvotun oppimisen ongelmaksi, erityisesti monivaiheisessa ennustamisessa. Tämä lähestymistapa muodostaa syöte-tulospareja, joissa syöte koostuu kiinteästä määrästä aiempia havaintoja ja tulos sisältää ennustettavan tulevien arvojen sekvenssin. Tämä tekniikka mahdollistaa mallien oppivan ajallisia riippuvuuksia ja tarkkojen monivaiheisten ennusteiden tekemisen.

Tässä työssä data on sekvensoitu 60 päivän pituisiin sekvensseihin siten, että ensimmäinen sekvenssi on $[\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{60}]$, missä $\mathbf{x}_i$ on vektori, joka sisältää kaikki malliin otetut tekniset indikaattorit. Tulosteena käytetään 20 seuraavan päivän arvoja. Tuloste on muotoa $[y_1, y_2, \dots, y_{20}]$, missä y_i on i :nnen päivän logaritminen palutus arvo. Sekä mallien koulutuksessa, että mallien testauksessa on käytetty liukuvan ikkunan menetelmää (Chou & Nguyen, 2018), (Bhatt ym., 2022). Syöteen ja vasteen ”ikkuna” siirtyy yhden päivän eteenpäin aina seuraavalle syötteelle kuten kuvassa



Kuva 4.4. MAPE muutos eri muuttujien lukumäärällä Applen osakedatassa.

4.5.

4.4 Mallien koulutus ja testaus

Data on jaettu koulutus ja testaus dataan siten, että 90% datasta kuuluu koulutus dataan ja 10% testaus dataan. Datan jakautuu kahteen osaan ajan perusteella. Koulutusdata sisältää datan vanhimmat datapisteet ja testausdata tuoreimman 10% datasta.

Jotta testausdatasta saadaan mahdollisimman paljon tulosteita, niin mallien testaus suoritetaan liukuvan ikkunan menetelmällä, jossa uusi 20 päivän ennuste tehdään aina päivä kerrallaan, kunnes kaikille mahdollisille päiville on saatu ennuste tehtyä, kuten kuvassa 4.5. Malleilla, joissa käytetään 60 päivän sekvenssiä, ei voida laskea ennustetta ensimmäiselle 60 päivälle, sillä malli tarvitsee ne pisteet syöteinä. Malleja arvioidaan kolmen erilaisen virheen avulla: keski neliövirhe MSE (Mean Squared Error), keskivirhe RMSE (Root Mean Squared Error) ja prosentuaalinen absoluuttinen keskivirhe MAPE (Mean Absolute Percentage Error). Jokaiselle 20 päivän ennusteelle on laskettu virhearvo. Näistä kaikista virheistä on raportoitu keskiarvot. Se kertoo, miten virhe keskimääräisesti käyttäytyy kaikkien 20 päivän sekvenssien välillä.

MSE saadaan laskettua kaavalla:

$$(4.18) \quad MSE = \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}$$

1. syöte-	$x_1 x_2 x_3 x_4 \dots x_{59} x_{60} x_{61} x_{62} x_{63} x_{64} \dots$
vaste pari	$y_{61} y_{62} y_{63} \dots y_{79} y_{80} y_{81} y_{82} y_{83} y_{84} \dots$
2. syöte-	$x_1 x_2 x_3 x_4 \dots x_{59} x_{60} x_{61} x_{62} x_{63} x_{64} \dots$
vaste pari	$y_{61} y_{62} y_{63} \dots y_{79} y_{80} y_{81} y_{82} y_{83} y_{84} \dots$
3. syöte-	$x_1 x_2 x_3 x_4 \dots x_{59} x_{60} x_{61} x_{62} x_{63} x_{64} \dots$
vaste pari	$y_{61} y_{62} y_{63} \dots y_{79} y_{80} y_{81} y_{82} y_{83} y_{84} \dots$

Kuva 4.5. Kolme ensimmäistä syöte vaste paria. Syöte on merkitty sinisellä ja vaste punaisella. Numero kertoo päivän, esimerkiksi 1. syöte on päivät 1-60 ja sille vasteet ovat päiviltä 61-80.

RMSE saadaan laskettua kaavalla:

$$(4.19) \quad RMSE = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}}$$

MAPE saadaan laskettua kaavalla:

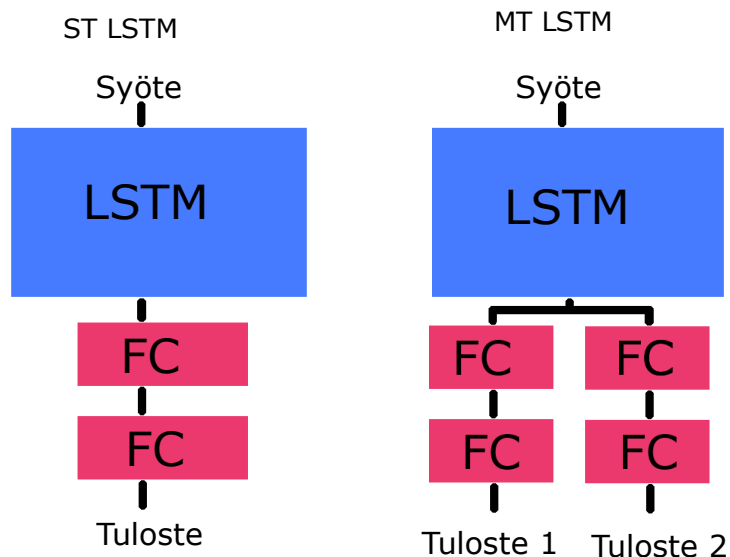
$$(4.20) \quad MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

5 Mallien sovittaminen Applen osake dataan

Vertailtavia malleja on yhteensä 5 kappaletta. Kolme mallia, jotka ennustavat ainoastaan 20 seuraavan päivän logaritmista palautusarvoa ovat SVR, LSTM ja Transformer. Näiden lisäksi monitehtävä tilanteeseen on sovellettu LSTM ja Transformer malleja. SVR on suhteellisen hyvä perusmalli, johon muita monimutkaisempia malleja verrataan. Sillä on mainio kyky yleistää ja sen tarkkuus on hyvä erilaisille datoille. Jotta monimutkaisemmat mallit voivat olla käyttökelpoisia, on niiden myös päihitettävä yksinkertaisemmat mallit. Ei ole kannattavaa tehdä monimutkaista mallia, jos yksinkertaisempi malli pystyy parempaan vähemmällä työllä. SVR mallia on käytetty pörssidatan mallintamiseen muun muassa lähteessä (Guo ym., 2018).

SVR malli eroaa hieman muista malleista siten, että sille syötettävä sekvenssi on lyhyempi. Useiden tekemieni testien perusteella suurella sekvenssillä SVR menettää tarkkuuttaan ja antaa huonompia tuloksia. Sekvenssin pituudeksi valittiin 15 päivää. Malli ennustaa liikkuvan ikkunan menetelmällä siten, että ensimmäisen arvon se ennustaa käyttäen 15 edellistä päivää. Seuraavalle ennustuskohdalle se siirtyy yhden päivän eteenpäin, jolloin ensimmäinen ennustettu arvo on syötesekvenssin viimeinen arvo. Näin siirrytään eteenpäin, kunnes kaikki 20 tulevaa arvoa on saatu ennustettua.

ST LSTM mallissa on samanlainen LSTM blokki, kuin mitä kohdassa 3.6 on esitelty. Siinä on yksi kerros ja piilotetun kerroksen koko on 4. LSTM blokin perässä on kaksi täysin kytkettyä kerrosta. Malli antaa tulosteena 20 seuraavan päivän ennusteen logaritmiselle palautusarvolle.



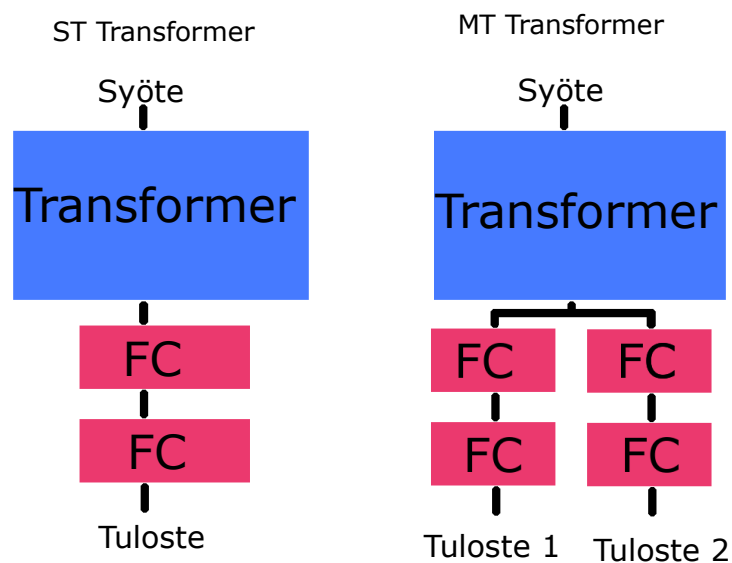
Kuva 5.1. LSTM ja monitehtävä LSTM mallit

MT LSTM malli on rakenteeltaan samanlainen kuin ST malli, mutta logarit-

misen palautusarvon lisäksi se ennustaa myös ATR indikaattoria. Näille kahdelle tehtävälle yhteistä mallissa on LSTM osuus. Molemmilla tehtävillä on lisäksi kaksi tehtäväkohtaista täysin kytkettyä kerrosta. Esimerkki mallista löytyy kuvasta 5.1.

ST Transformer mallissa on 2 monihaaraista tarkkaavaisuutta, 2 päällekkäistä transformer kerrosta, piilotettujen kerrosten dimensio on 4, eteenpäin syöttävän kerroksen dimensio on 4 ja dropout arvo on 0.1. Transformerin perässä on kaksi täysin kytkettyä kerrosta samalla tavalla kuin LSTM mallissa. Mallin tulosteena on 20 seuraavan päivän logaritminen palautusarvo.

MT Transformer malli on rakenteeltaan samanlainen kuin ST malli, mutta kuten MT LSTM mallissa se ennustaa myös ATR indikaattoria. Molemmilla tehtävillä on tehtäväkohtaiset kerrokset, jotka koostuvat kahdesta täysin kytketystä kerroksesta. Yhteinen osuus on Transformer osuus kuten kuvassa 5.2.



Kuva 5.2. Transformer ja monitehtävä Transformer mallit

5.1 Tulosten vertailua

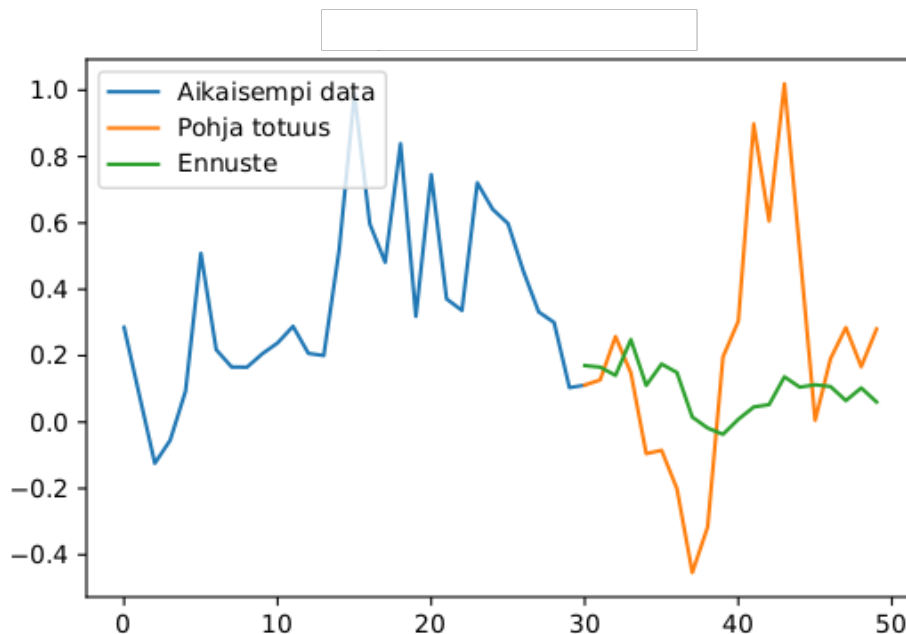
Taulukossa 5.1 on laskettu kolme eri virhe arvoa kaikille malleille: keskivirhe MSE (eng. Mean Squared Error), juuri keskivirhe RMSE (eng. Root Mean Squared Error) ja keskimääräinen absoluuttinen prosentuaalinen virhe MAPE (eng. Mean Absolute Percentage Error). Mallit ovat melko lähellä toistensa arvoja. Pienimmät virheet kaikissa virhemittauksissa on Monitehtävä Transformerilla. MT Transformerin MSE on 0.1587 ja seuraavaksi pienin MSE on MT LSTM mallilla 0.1608. Suurimmat MSE ja RMSE virheet olivat ST LSTM mallilla 0.1928 ja 0.4060. Suurin MAPE on ST Transformerilla 3.5327. Yksinkertainen SVR mallin virheet ovat pienemmät kuin ST LSTM mallin. RMSE ja MAPE ovat pienempiä kuin ST Transformerilla ja MSE on vain hieman suurempi SVR mallilla kuin ST Transformerilla.

Taulukko 5.1. Mallien virhearvoja

Malli	MSE	RMSE	MAPE
SVR	0.1891	0.3882	2.7921
ST LSTM	0.1928	0.4060	3.0955
MT LSTM	0.1608	0.3709	2.6632
ST Transformer	0.1856	0.3994	3.5327
MT Transformer	0.1587	0.3694	2.2898

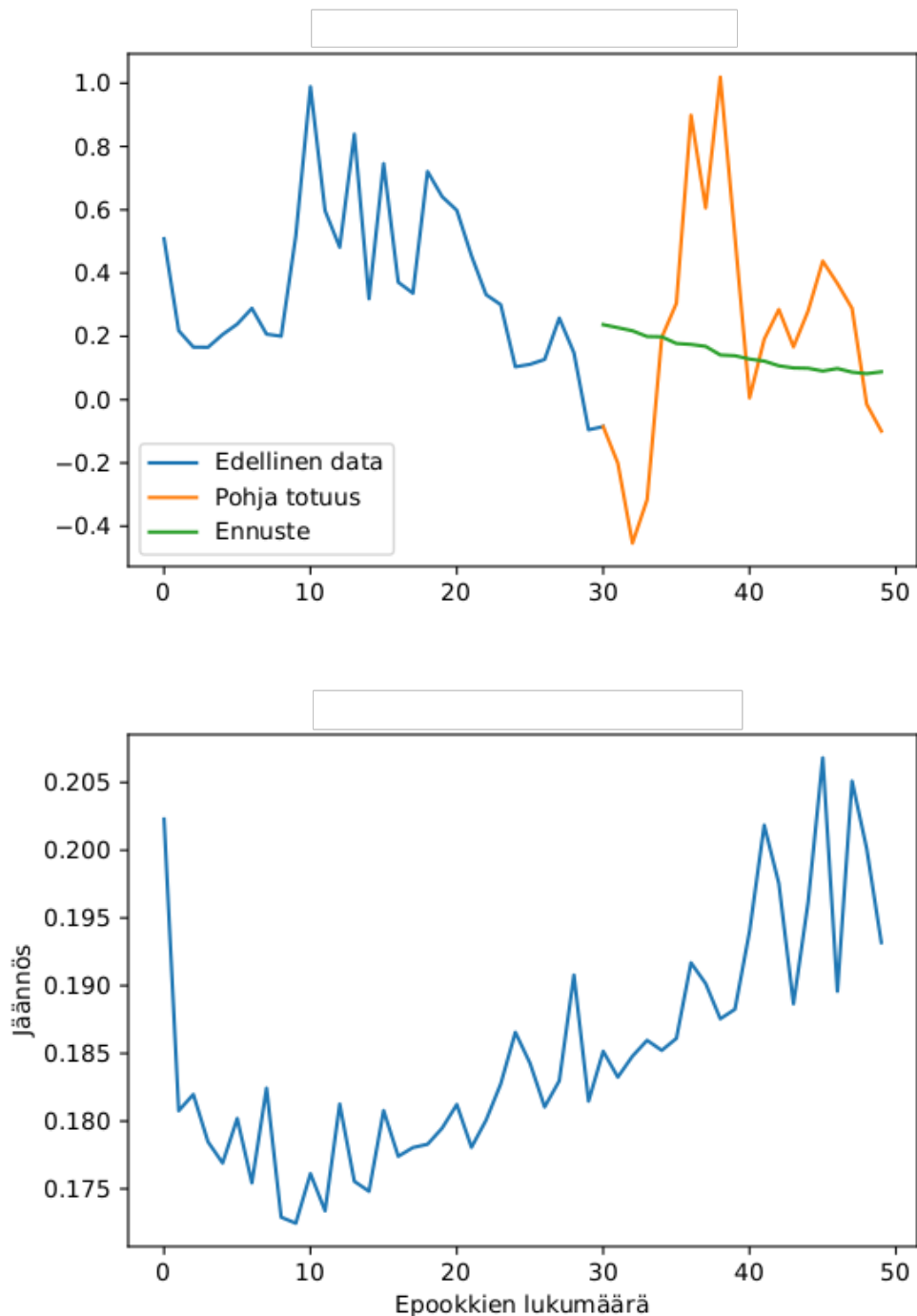
Kuvassa 5.3 näkyy yksi useasta 20 päivän ennusteesta, joiden perusteella mallille on laskettu virhearvot taulukossa 5.1. Se näyttää mukailevan pohjatotuutta, eikä poikkeaa siitä huomattavasti. Muutokset logaritmiseen palautusarvoon näyttävät tulevan hieman viiveellä verrattuna pohjatotuuteen.

Kuvassa 5.4 näkyy 20 päivän ennuste logaritmiselle palautusarvolle. Kuvassa 5.4 näkyy 20 päivän ennuste logaritmiselle palautusarvolle



Kuva 5.3. 20 päivän ennuste käyttäen SVR mallia.

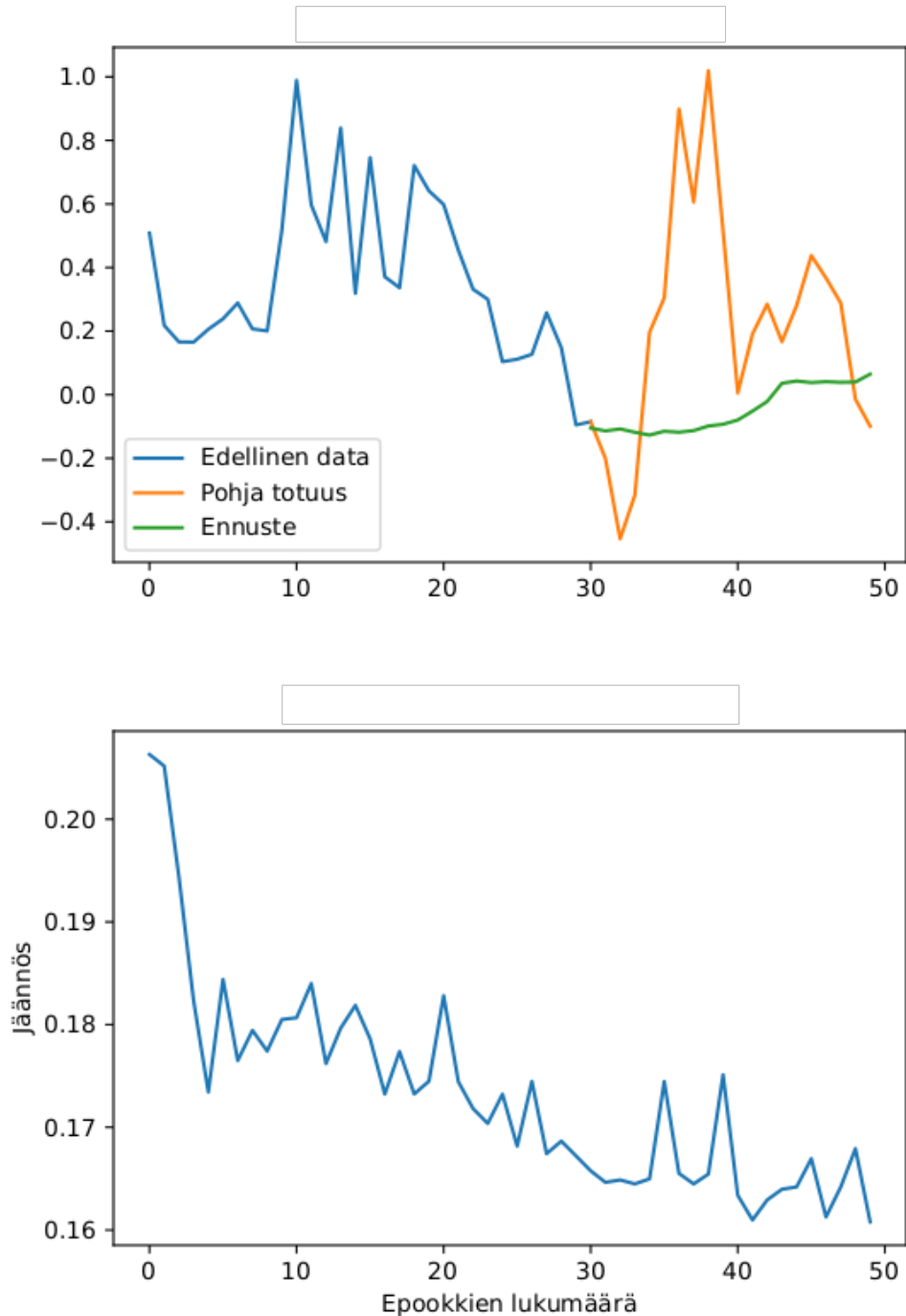
Kuvassa 5.4 näkyy 20 päivän ennuste logaritmiselle palautusarvolle. Malli näyttää oppineen lähinnä trendin koulutus datasta. Mallissa ei näy suurta vaihtelua, vaan se kulkee miltei suoraan pohjatotuuden läpi juurikaan mukailematta sen vaihtelua. Kuvassa 5.4 alhaalla näkyy miten mallin jäännös on käyttäytynyt sitä kouluttaessa. Alussa se on lähtenyt laskemaan kuten olettaa saattaa, mutta ennen 10 epookkia sen suunta on lähtenyt nousuun. Paras malli siis tulisi noin 9 epookin kohdalla, jonka jälkeen se on alkanut ylioppimaan. Tämän mallin kohdalla olisi hyvä käyttää aikaista pysäytystä (eng. early stopping), joka pysäyttäisi mallin koulutuksen aikaisemmin ja säilyttäisi parhaan mahdollisen mallin. Tällä menetelmällä MSE olisi jossain 0.17 ja 0.175 välillä.



Kuva 5.4. Ylhäällä 20 päivän ennuste käyttäen yhden tehtävän LSTM mallia. Alhaalla mallin virhe 50 koulutus kierroksen ajalta.

Kuvassa 5.5 ylhäällä näkyy yksi kahden tehtävän LSTM mallin tulosteista. Siinä näkyy hieman enemmän vaihtelua verrattuna yhden tehtävän malliin. Se ei kuitenkaan vaihtelee aivan yhtä paljon, kuin SVR malli, vaan yleistää hieman enemmän. Samassa kuvassa alhaalla nähdään, miten jäännös on käyttäytynyt koulutuksen aikana. Alun suuren pudotuksen jälkeen oppiminen on hieman tasaantunut. Siitä huolimatta, että virhe välillä kasvaa, trendi on kuitenkin vahvasti alas päin. Mitä enemmän mallia

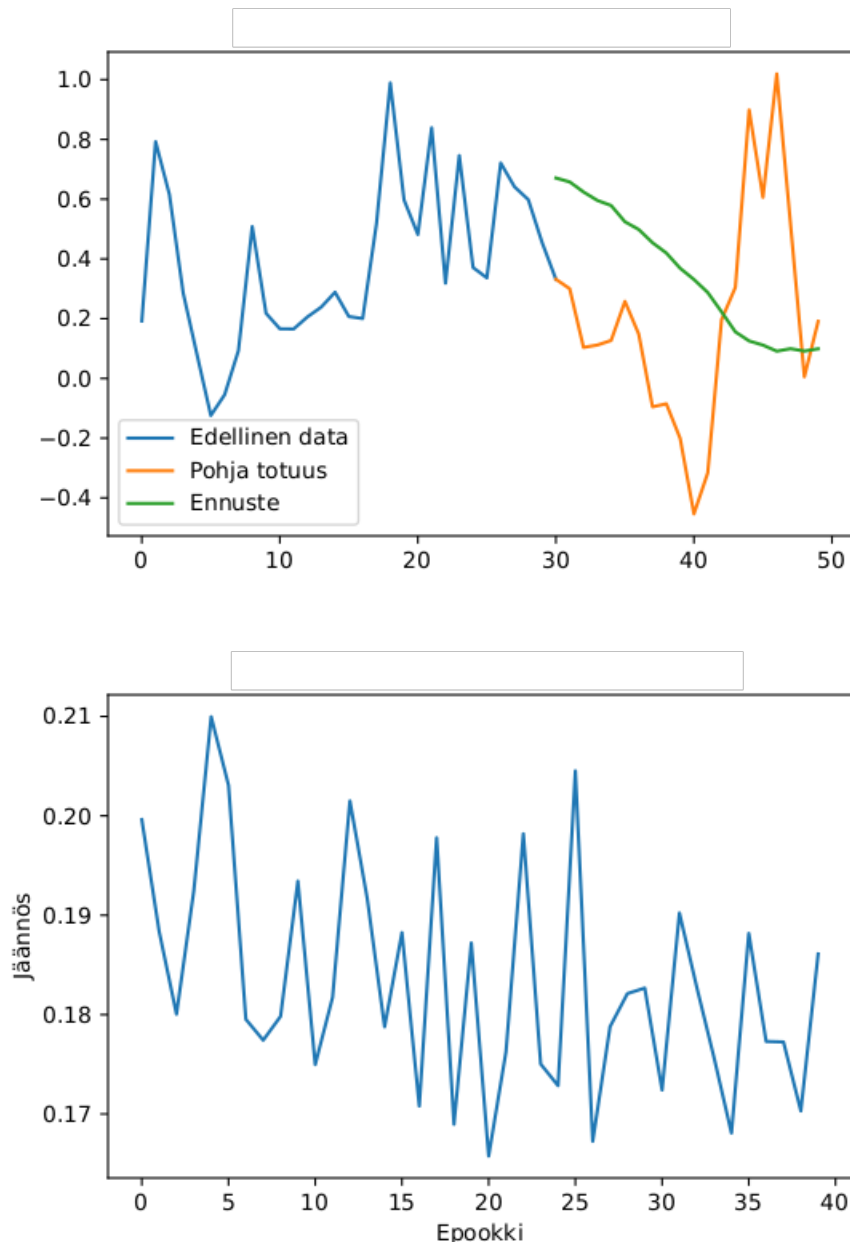
on koulutettu, sitä paremmin se on oppinut. On mahdollista, että lisää kouluttamalla saataisiin vielä parempia tuloksia. Mallin oppimisen kannalta käyrä näyttää hieman mukailevan pohjatotuutta ja voidaan sanoa, että malli on oppinut 50 epookin aikana.



Kuva 5.5. Ylhäällä 20 päivän ennuste käyttäen kahden tehtävän LSTM mallia. Alhaalla mallin virhe 50:n koulutus kierroksen ajalta.

Kuvassa 5.6 ylhäällä näkyy yhden tehtävän Transformer mallin ennuste 20 seuraavalle päivälle. Ennusteen suunta on oikea, mutta se on melko kaukana todellisista

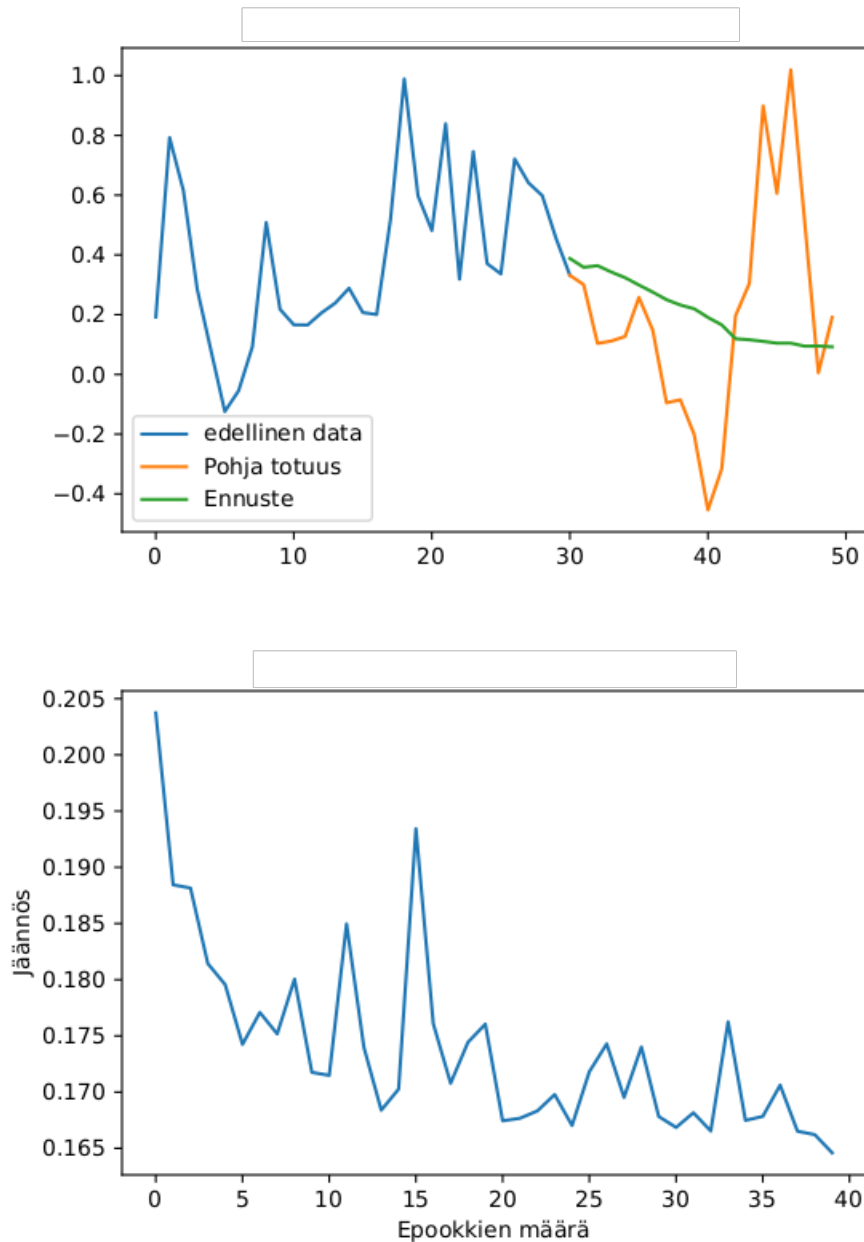
arvoista. Samassa kuvassa alempana näkyy, miten virhe on käyttäytynyt koulutusai-
kana. Kuvassa virhe kulkee ylös alas miltei ilman suuntaa. Se viittaa vahvaan yliop-
pimiseen. Tämä voi johtua siitä, että oppimisnopeus on ollut kouluttaessa liian suuri.
Siitä johtuen malli ei ole löytänyt hyviä parametriarvoja, vaan on aina hypännyt liian
kauas niistä. Mallissa oppimisnopeus on ajan myötä pienentynyt. Huiput näyttävät
pienenevän hieman loppua kohden ja virhe näyttäisi jäävän välille $[0.17, 0.19]$ jos
mallia olisi koulutettu useampi kierros. Tästä ei voida olla kuitenkaan varmoja, sillä
virhe voi lähteä myös nousuun, kun mallia koulutetaan liian kauan.



Kuva 5.6. Ylhäällä 20 päivän ennuste käyttäen yhden tehtävän Transformer mallia. Alhaalla mallin virhe 40:n koulutus kierroksen ajalta.

Kuvassa 5.7 ylhäällä näkyy kahden tehtävän Transformerin ennuste 20:lle seu-

raavalle päivälle. Se näyttäisi mukailevan todellista käytöstä hieman lähemmin kuin yhden tehtävän mallille. Näiden käyrien muodot ovat melko samankaltaiset, mutta kahden tehtävän malli kulkee paremmin datan läpi. Samassa kuvassa alhaalla nähdään, kuinka virhe on käyttäytynyt koulutusajalla. Noin 10–15 epookin välillä on oppiminen lähtenyt väärään suuntaan, mutta sen jälkeen se on tasaantunut hyvin ja lähtenyt melko tasaiseen laskuun. Tämä kuvaa sitä, että malli on oppinut melko hyvin eikä suurempaa ylioppimista ole tapahtunut.



Kuva 5.7. Ylhäällä 20 päivän ennuste käyttäen kahden tehtävän Transformer mallia. Alhaalla mallin virhe 40:n koulutus kierroksen ajalta.

Mallien koulutuksessa on havaittavissa jonkinasteista ylisovittamista. On mahdollista pysäyttää koulutus, kun huomataan mallin ylisovittavan. Tätä varten koulutusda-

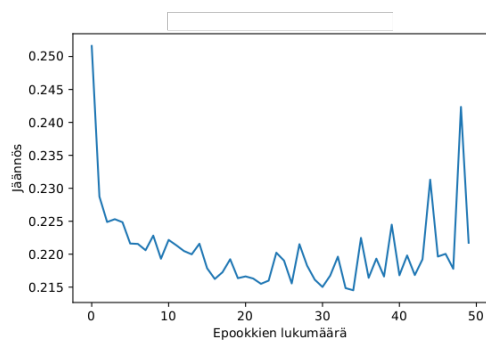
ta pitää jakaa vielä erikseen koulutus- ja validaatiodataan. Koulutusdatan viimeiset 1000 tapausta positetaan koulutusdatasta ja siirretään validaatiodataan. Validaatiodatalla arvioidaan mallin sopivuutta jokaisen koulutuskierroksen jälkeen. Näiden mallien vertailu taulukossa 5.2. Aikaisella pysäytyksellä on myös vaarana se, että malli sovituu liian hyvin validaatiodataan, mutta ei enää testaus dataan.

Aikaisen pysäytyksen tapauksessa usean tehtävän mallit antoivat parempia tuloksia kuin yhden tehtävän mallit. Pienimmät MSE ja RMSE arvot olivat MT Transformerilla ja pienin MAPE oli MT LSTM mallilla. Transformer mallilla mmonitehtävä asetelma lisäksi paransi mallin tuloksia huomattavasti, sillä suurimmat MSE, RMSE ja MAPE arvot olivat ST Transformerilla. Jokainen malli toimi huonommin verrattuna versioon, jossa koko koulutusdata on käytetty mallin koulutukseen. Parhaankin mallin suorituskky paranee vain hieman verrattuna taulukon 5.1 SVR malliin.

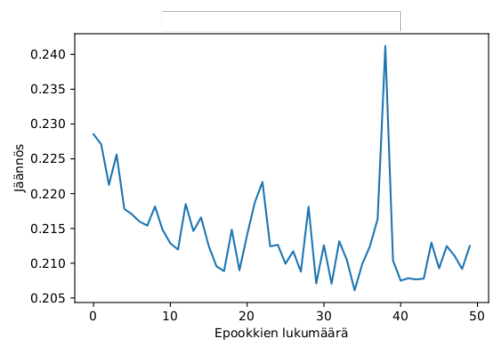
Tuloksista voi päätellä, että kaikilla malleilla uusimmat pisteet vaikuttavat mallin tarkkuuteen merkittävästi. Suurin ero kuitenkin on LSTM malleilla, jotka huononivat suhteessa eniten, kun tuorein data käytetään validaatioon. Kuvista 5.8 - 5.11 nähdään, että kaikki mallien validaatiot asettuvat melko samoille lukemille, mutta MT Transformer näyttäisi oppivan kaikista tasaisimmin. MT Transformer mallissa validaation virhe ei hypi edestakaisin merkittävästi, vaan laskeutuu hyvin tasaisesti ja pysyy alhaalla koko koulutuksen ajan. Transformerin tapauksessa täydentävä tehtävä parantaa tuloksia kaikista eniten. ST Transformerin tulokset olivat vertailuryhmän huonoimmat kun taas MT Transformerin tulokset olivat huomattavasti paremmat.

Taulukko 5.2. Mallien virhearvoja, missä koulutusdata on jaettu koulutus- ja validaatiodatoin

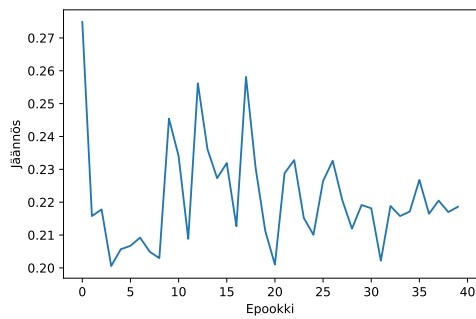
Malli	MSE	RMSE	MAPE
ST LSTM	0.2371	0.4494	3.0430
MT LSTM	0.2106	0.4262	2.9298
ST Transformer	0.3353	0.5293	3.1196
MT Transformer	0.1726	0.3846	3.0838



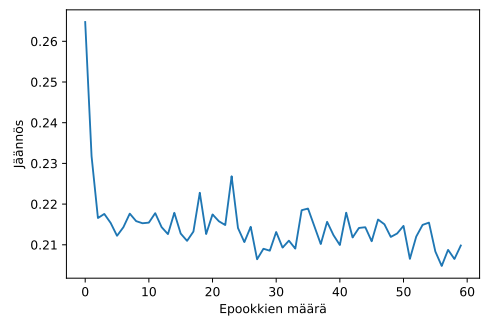
Kuva 5.8. ST LSTM validaatio



Kuva 5.9. MT LSTM validaatio



Kuva 5.10. ST Transformer validaatio



Kuva 5.11. MT Transformer validaatio

6 Pohdinta

6.1 Tulosten yhteenveto ja jatkotutkimus ehdotukset

Tässä työssä olen tutkinut erilaisten menetelmien sopivuutta pörssidatan mallintamiseen. Työn painopisteenä on etenkin monitehtäväoppimisen vaikutus mallin tarkkuuteen ja sen koulutukseen. Malleista on muodostettu sekä yhden tehtävän mallit, että monitehtävämallit, joita on vertailtu niin keskenään kuin vertailukohta mallin kanssa. Tämän takia tutkimuskysymyksiksi valikoitui kaksi kysymystä:

1. Miten monitehtävämallit pärjäävät vertailussa yhden tehtävän mallien ja vertailukohtamallin kanssa?
2. Miten monitehtäväoppiminen vaikuttaa mallien kouluttamiseen?

Yhdentehtävän mallit eivät pärjänneet paremmin kuin yksinkertainen SVR malli. Ne yli sovittivat todella helposti ja antoivat näin ollen epäluotettavia tuloksia. Yhtenä syynä tälle voi olla se, että muuttujia on monta. Tämä aiheuttaa sen, että parametrejä syntyy LSTM ja Transformer malleihin paljon verrattuna monikerroksiseen perseptron verkkoon, jolla muuttujien valinta on suoritettu. Näillä muuttujamäärillä mallit eivät ole juuri käyttökelpoisia, vaan ainakin muuttujien määrää olisi pudotettava. Jos aikaa on tarpeeksi, niin eteenpäin valinta olisi hyvä suorittaa käyttäen virheen laskemiseen LSTM ja Transformer verkkoja. Näiden verkkojen koulutus vie paljon aikaa verrattuna MLP verkkoon ja siksi ei ole tämän työn kannalta järkevää toteuttaa jokaiselle verkolle muuttujien valintaa erikseen.

Aikaisemmissa tutkimuksissa, kuten (Park ym., 2022) ja (Prabhu ym., 2021) on huomattu, että monitehtävämallit tuottavat parempia tuloksia kuin yhden tehtävän mallit, jos tehtävät liittyvät toisiinsa. Tämän työn löydökset ovat myös linjassa näiden aikaisempien tutkimusten kanssa. Kaikki malleille lasketut virhearvot pienenevät yhden tehtävän malleihin verrattuna. Nämä mallit eivät ainoastaan parantuneet yhden tehtävän malleista vaan päihittivät myös SVR mallin. Mielenkiintoista on se, että vaikka mallit pysyvät miltei samana lukuun ottamatta kahta FC lisäkerrosta ja toista tehtävää, niin ylioppiminen parani huomattavasti monitehtävämalleilla. Näiden tulosten perusteella useampi tehtävä auttaa enemmän, kuin mallin vaihtaminen monimutkaisempaan. Lisäksi tulokset viittaavat siihen, että LSTM ja Transformer malleilla ei välttämättä tarvita dimension pienennystä ylioppimis- tilanteissa, vaan ongelma saattaa ratketa monitehtävä menetelmillä. Jos päätehtävälle pystytään löytämään toinen tehtävä, joka on jollain tavalla lähellä päätehtävää, saattaa mallin ylisovitus ja tulokset parantua.

Sekä LSTM, että Transformer mallit näyttäisivät vaativan uusimman datan saadakseen parhaita tuloksia. Mallit, joissa koulutusdatasta oli siirretty validaatiodataan 1000 viimeistä mittausta suoriutuivat huomattavasti huonommin kuin mallit, jotka koulutettiin koko koulutusdatalla. Tästä voidaan päätellä, että uusin data sisältää merkittävästi mallille tärkeitä informaatiota. Myös aikaisemmissa tutkimuksissa on päästy samankaltaisiin tuloksiin. Esimerkiksi Wheelwright ym. (1998) painottavat

viimeisimmän datan merkitystä aikasarjadatan ennustamisessa erityisesti epävakaila markkinoilla kuten pörssimarkkinoilla. He sanovat, että rahoitusmarkkinat ovat hyvin dynaamisia, ja uusimmat datapisteet sisältävät usein tärkeintä tietoa tulevien trendien ennustamiseen.

Mahdollinen jatko tutkimus voisi selvittää, mitä tapahtuu ylisovitukselle ja mallin tuloksille, jos malliin ottaa useamman kuin kaksi tehtävää. Mielenkiintoista olisi tietää, paraneeko ylisovittaminen entisestään ja saadaanko sillä vielä parempia tuloksia. Jos kaksi tehtävää ovat aivan erilaisia eivätkä käyttäydy yhtään samalla tavalla voi mallin tulos huonontua monitehtävätilanteessa. Mielenkiintoista olisi myös se, miten tehtävien yhteen liittyvyys vaikuttaa mallin tarkkuuteen ja saadaanko enemmän yhteen liittyvillä tehtävillä parempia tuloksia kuin vähemmän yhteen liittyvillä. Lisäksi olisi myös hyvä selvittää, saadaanko datasta opittua mitkä tehtävät olisivat hyvät yhdessä ja mitkä eivät sovi samaan malliin.

Tutkimuksessa olisi voinut kokeilla kouluttaa malleja hieman pidemmälle ja suorittaa koulutuksessa simuloitu hehkutus, jonka avulla olisi todennäköisemmin löydetty globaali minimi mallin jäännökselle. Näin mallien tulosteet olisivat voineet olla vielä paremmin vertailukelpoisia keskenään. Kuitenkin näistä tuloksista nähdään hyvin, miten erilaiset mallit käyttäytyvät kyseisen datan kanssa.

6.2 Luotettavuus ja eettisyys

Tutkimusta tehdessäni olen keskittynyt Tutkimuseettisen neuvottelukunnan (TENK) hyvään tieteelliseen käytäntöön (HTK). TENKin mukaan hyvän tieteellisen käytännön peruspilareita ovat luotettavuus, rehellisyys, arvostus ja vastuunkanto. Tutkielmani luotettavuuden olen varmistanut muun muassa käyttämällä tunnettuja ja hyväksi todettuja tutkimusmenetelmiä. Rehellisyyden olen varmistanut raportoimalla kaikki tulokseni totuudenmukaisesti ja kuten ne ovat minulle osoittautuneet. Olen pitänyt arvostuksesta huolta viittaamalla muihin tutkimuksiin asianmukaisella tavalla, enkä ole plagioinut tekstiä suoraan lähteestä. Vastuunkanto täyttyy sillä, että koko tutkimus on omaa käsialaani ja olen suorittanut sen kirjoittamisen kokonaan itse. Olen saanut neuvoa ja vinkkejä muilta, mutta päätökset ja toteutus on kokonaan minun. Tekoälyä olen tutkimuksessani käyttänyt kirjoitusasun korjaamiseen, lähteiden etsimiseen ja ohjelmien koodin vikojen korjaamiseen.

Lähteet

- Apple (AAPL) Stock Dataset 1980-2025 — kaggle.com [[Accessed 17-02-2025]]. (ei julkaisupäivää).
- Awad, M., Khanna, R., Awad, M., & Khanna, R. (2015). Support vector regression. *Efficient learning machines: Theories, concepts, and applications for engineers and system designers*, 67–80.
- Bhatt, A., Ongsakul, W., Singh, J. G., ym. (2022). Sliding window approach with first-order differencing for very short-term solar irradiance forecasting using deep learning models. *Sustainable Energy Technologies and Assessments*, 50, 101864.
- Bousqaoui, H., Slimani, I., & Achchab, S. (2021). Comparative analysis of short-term demand predicting models using ARIMA and deep learning. *International Journal of Electrical and Computer Engineering*, 11(4), 3319.
- Brownlee, J. (2018). *Deep learning for time series forecasting: predict the future with MLPs, CNNs and LSTMs in Python*. Machine Learning Mastery.
- Caruana, R. (1997). Multitask learning. *Machine learning*, 28, 41–75.
- Chen, P.-C., Tsai, H., Bhojanapalli, S., Chung, H. W., Chang, Y.-W., & Ferng, C.-S. (2021). A simple and effective positional encoding for transformers. *arXiv preprint arXiv:2104.08698*.
- Chirag. (2024). Understanding multiclass classification using SVM. <https://www.analyticsvidhya.com/blog/2021/05/multiclass-classification-using-svm/>
- Chou, J.-S., & Nguyen, T.-K. (2018). Forward forecast of stock price using sliding-window metaheuristic-optimized machine-learning regression. *IEEE Transactions on Industrial Informatics*, 14(7), 3132–3142.
- Cilimkovic, M. (2015). Neural networks and back propagation algorithm. *Institute of Technology Blanchardstown, Blanchardstown Road North Dublin*, 15(1).
- Cohen, S. (2021). The basics of machine learning: strategies and techniques. *Teoksessa Artificial intelligence and deep learning in pathology* (s. 13–40). Elsevier.
- Cortes, C. (1995). Support-Vector Networks. *Machine Learning*.
- Crawshaw, M. (2020). Multi-task learning with deep neural networks: A survey. *arXiv preprint arXiv:2009.09796*.
- Dongare, A., Kharde, R., Kachare, A. D., ym. (2012). Introduction to artificial neural network. *International Journal of Engineering and Innovative Technology (IJEIT)*, 2(1), 189–194.
- Du, K.-L., & Swamy, M. N. (2013). *Neural networks and statistical learning*. Springer Science & Business Media.
- Graves, A., & Graves, A. (2012). Long short-term memory. *Supervised sequence labelling with recurrent neural networks*, 37–45.
- Guo, Y., Han, S., Shen, C., Li, Y., Yin, X., & Bai, Y. (2018). An adaptive SVR for high-frequency stock price forecasting. *IEEE Access*, 6, 11397–11404.
- Hudson, R. S., & Gregoriou, A. (2015). Calculating and comparing security returns is harder than you think: A comparison between logarithmic and simple returns. *International Review of Financial Analysis*, 38, 151–162.

- Jain, S. M. (2022). *Introduction to Transformers for NLP : With the Hugging Face Library and Models to Solve Problems* (1st ed. 2022.). Apress.
- Kaski, S., & Peltonen, J. (2007). Learning from relevant tasks only. *Machine Learning: ECML 2007: 18th European Conference on Machine Learning, Warsaw, Poland, September 17-21, 2007. Proceedings 18*, 608–615.
- Khan, A. H., Shah, A., Ali, A., Shahid, R., Zahid, Z. U., Sharif, M. U., Jan, T., & Zafar, M. H. (2023). A performance comparison of machine learning models for stock market prediction with novel investment strategy. *Plos one*, 18(9), e0286362.
- Kumar, G., Jain, S., & Singh, U. P. (2021). Stock market forecasting using computational intelligence: A survey. *Archives of computational methods in engineering*, 28(3), 1069–1101.
- Laakkonen, H. (2023). Yliopistoissa on jo huomattu opiskelijoiden käyttäneen tehtävissä Chat GPT:tä – oppilaitokset suhtautuvat silti tekoälyn käyttöön myönteisesti — uusimaa.fi [[Accessed 06-01-2025]]. <https://www.uusimaa.fi/uutissuomalainen/5800768>
- Li, W., Liu, C., Hu, C., Niu, C., Li, R., Li, M., Xu, Y., & Tian, L. (2024). Application of a hybrid algorithm of LSTM and Transformer based on random search optimization for improving rainfall-runoff simulation. *Scientific Reports*, 14(1), 11184.
- Lu, Y., Kumar, A., Zhai, S., Cheng, Y., Javidi, T., & Feris, R. (2017). Fully-adaptive feature sharing in multi-task networks with applications in person attribute classification. *Proceedings of the IEEE conference on computer vision and pattern recognition*, 5334–5343.
- Lu, Y., & Salem, F. M. (2017). Simplified gating in long short-term memory (lstm) recurrent neural networks. *2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS)*, 1601–1604.
- Mao, K. Z. (2004). Orthogonal forward selection and backward elimination algorithms for feature subset selection. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 34(1), 629–634.
- Marcano-Cedeño, A., Quintanilla-Domínguez, J., Cortina-Januchs, M., & Andina, D. (2010). Feature selection using sequential forward selection and classification applying artificial metaplasticity neural network. *IECON 2010-36th annual conference on IEEE industrial electronics society*, 2845–2850.
- Medsker, L. R., Jain, L., ym. (2001). Recurrent neural networks. *Design and Applications*, 5(64-67), 2.
- Mirkin, B. (2011). Choosing the number of clusters. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 1(3), 252–260.
- Murphy, J. J. (1999). *Technical analysis of the financial markets: A comprehensive guide to trading methods and applications*. Penguin.
- Padial, D. L. (2018). Technical Analysis Library in Python [Accessed: 2025-01-13].
- Panna, M. (2017). Note on simple and logarithmic return. *APSTRACT: applied studies in agribusiness and commerce*, 11, 127–136.
- Park, H. J., Kim, Y., & Kim, H. Y. (2022). Stock market forecasting using a multi-task approach integrating long short-term memory and the random forest framework. *Applied Soft Computing*, 114, 108106.

- Patel, J., Shah, S., Thakkar, P., & Kotecha, K. (2015). Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques. *Expert systems with applications*, 42(1), 259–268.
- Prabhu, D., Chhabra, A., Goyal, S., & Das, B. (2021). Multi task learning for financial forecasting. *Emerging Technologies in Data Mining and Information Security: Proceedings of IEMIS 2020, Volume 3*, 397–411.
- Pring, M. J. (2002). *Study guide for technical analysis explained* (Vol. 5). McGraw-Hill New York.
- Ryll, L., & Seidens, S. (2019). Evaluating the performance of machine learning algorithms in financial market forecasting: A comprehensive survey. *arXiv preprint arXiv:1906.07786*.
- Salehinejad, H., Sankar, S., Barfett, J., Colak, E., & Valaee, S. (2017). Recent advances in recurrent neural networks. *arXiv preprint arXiv:1801.01078*.
- Schmidhuber, J. (2022). Annotated history of modern ai and deep learning. *arXiv preprint arXiv:2212.11279*.
- Shumway, R. H., Stoffer, D. S., Shumway, R. H., & Stoffer, D. S. (2017). ARIMA models. *Time series analysis and its applications: with R examples*, 75–163.
- Sugali, K., Sprunger, C., & Inukollu, V. N. (2024). AI TESTING: ENSURING A GOOD DATA SPLIT BETWEEN DATA SETS (TRAINING AND TEST) USING K-MEANS CLUSTERING AND DECISION TREE ANALYSIS. *International Journal on Soft Computing*, 34.
- Tuominen, H. (2020). Johdatus tekoälyn taustalla olevaan matematiikkaan.
- Vartiainen, H., Tedre, M., Jormanainen, I., Kahila, J., Valtonen, T., & Toivonen, T. (2021). Tekoäly, koneoppiminen ja teknologinen murros: Kohti datatoimijuutta ja tulevaisuuden design-taitoja. *Ainedidaktikka*, 5(2).
- Vaswani, A. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*.
- Wheelwright, S., Makridakis, S., & Hyndman, R. J. (1998). *Forecasting: methods and applications*. John Wiley & Sons.
- Zeyer, A., Bahar, P., Irie, K., Schlüter, R., & Ney, H. (2019). A comparison of transformer and lstm encoder decoder models for asr. *2019 IEEE automatic speech recognition and understanding workshop (ASRU)*, 8–15.
- Zhang, Y., & Yang, Q. (2018). An overview of multi-task learning. *National Science Review*, 5(1), 30–43.
- Zheng, J., Ramasinghe, S., & Lucey, S. (2021). Rethinking positional encoding. *arXiv preprint arXiv:2107.02561*.